



The Exact Security of BIP32 Wallets

Poulami Das
TU Darmstadt, Germany
poulami.das@tu-darmstadt.de

Andreas Erwig
TU Darmstadt, Germany
andreas.erwig@tu-darmstadt.de

Sebastian Faust
TU Darmstadt, Germany
sebastian.faust@tu-darmstadt.de

Julian Loss
University of Maryland, USA
lossjulian@gmail.com

Siavash Riahi
TU Darmstadt, Germany
siavash.riahi@tu-darmstadt.de

ABSTRACT

In many cryptocurrencies, the problem of key management has become one of the most fundamental security challenges. Typically, keys are kept in designated schemes called *wallets*, whose main purpose is to store these keys securely. One such system is the BIP32 wallet (Bitcoin Improvement Proposal 32), which since its introduction in 2012 has been adopted by countless Bitcoin users and is one of the most frequently used wallet system today. Surprisingly, very little is known about the concrete security properties offered by this system. In this work, we propose the first formal analysis of the BIP32 system in its entirety and without any modification. Building on the recent work of Das et al. (CCS '19), we put forth a formal model for hierarchical deterministic wallet systems (such as BIP32) and give a security reduction in this model from the existential unforgeability of the ECDSA signature algorithm that is used in BIP32. We conclude by giving concrete security parameter estimates achieved by the BIP32 standard, and show that by moving to an alternative key derivation method we can achieve a tighter reduction offering an additional 20 bits of security (111 vs. 91 bits of security) at no additional costs.

CCS CONCEPTS

• Security and privacy → Cryptography.

KEYWORDS

Wallets, cryptocurrencies, foundations, BIP32

ACM Reference Format:

Poulami Das, Andreas Erwig, Sebastian Faust, Julian Loss, and Siavash Riahi. 2021. The Exact Security of BIP32 Wallets. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS '21)*, November 15–19, 2021, Virtual Event, Republic of Korea. ACM, New York, NY, USA, 23 pages. <https://doi.org/10.1145/3460120.3484807>

1 INTRODUCTION

Decentralized cryptocurrencies such as Bitcoin or Ethereum have introduced a new digital payment paradigm which does not rely

on a central authority such as a bank or a credit card company. The main building block used in many popular cryptocurrencies to facilitate secure transfer and holding of assets are digital signatures. Loosely speaking, a user Alice in the system is identified by her public key pk_A which she uses as her address for receiving and sending payments. If Alice wants to send c coins of the underlying currency to another user Bob with address pk_B , she creates a transaction tx saying “Send c coins from pk_A to pk_B ” and signs tx using her secret key sk_A . She then uploads the transaction tx together with the signature σ to the public ledger (often also called blockchain) of the cryptocurrency. Once the tuple (tx, σ) is visible on the public ledger, the payment is completed meaning that now Bob owns an additional c coins of the underlying currency. Clearly, Alice’s funds remain secure only as long as no one can forge a signature σ on her behalf that verifies under pk_A . On top of this, it is generally recommended to use a fresh signing key for every new transaction stored on the public ledger to avoid that all transactions are linkable to the same user Alice. In the cryptocurrency space, the management and storage of secret keys is typically carried out by so-called *wallets* – which are pivotal for the security of cryptocurrency funds. Indeed, cryptocurrency wallets are a highly attractive target for hackers as illustrated by spectacular attacks against common cryptocurrency projects. For example, in 2018 alone, hackers managed to steal more than one billion USD worth of cryptocurrency from wallets [7, 8, 31].

While several recent works study the formal security properties of cryptocurrency wallets (see related work for a detailed discussion), one of the most widely used schemes – *the BIP32 wallet* [33] – has not been formally analyzed so far. This is somewhat surprising as BIP32 became a standard for deterministic Bitcoin wallets in 2012, and has been widely adopted since then (e.g., it is used in the deployment of popular wallets [1–3]). In this work, we address this gap and provide the first comprehensive study of the security properties achieved by the BIP32 wallet standard.

1.1 Deterministic Wallets

As we have already pointed out, to improve privacy it is important to not re-use the same signing key for too many public transactions. To explain why privacy is also beneficial for security, let us consider a user Alice who holds a single secret/public key pair (sk_A, pk_A) , and that she receives multiple payments to her address pk_A . As we have explained, such transactions contain her public key pk_A and are posted to the public ledger. Hence, an attacker can easily extract Alice’s balance via the public transaction ledger. Over time, pk_A ’s balance might grow, and at some point, the attacker

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CCS '21, November 15–19, 2021, Virtual Event, Republic of Korea

© 2021 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-8454-4/21/11...\$15.00

<https://doi.org/10.1145/3460120.3484807>

may identify pk_A as a high-priority target. The obvious approach to thwart an attacker's attempts of linking Alice's transactions would be for Alice to keep a set of l one-time random key pairs $\{(sk_1, pk_1), \dots, (sk_l, pk_l)\}$ within her wallet, where each key pair is used for a single transaction on the public ledger. However, this approach has the obvious downside of Alice having to store all of her keys on disk (as long as they still retain some amount of currency). This requires a lot of storage space and bears the risk of losing one of her keys, at which point the associated funds of that key are irrevocably lost. A simple approach to overcome these issues are *deterministic wallets*, proposed by Buterin [12]. A deterministic wallet usually contains a pair of master keys (msk , mpk) and a seed ch , which is also referred to as the *chaincode*. For every new transaction, the wallet deterministically derives a fresh session key pair (sk, pk) from the master keys with the help of deterministic key derivation algorithms. More precisely, the public key derivation algorithm takes as input the master public key mpk , the chaincode ch and an identifier ID and deterministically computes a one-time public key pk_{ID} . An analogous secret key derivation algorithm takes in msk , ch , and ID and deterministically computes a one-time secret key sk_{ID} that matches pk_{ID} (given that the arguments ch and ID in both derivations are identical). Going back to the example of BIP32, (msk, mpk) are generated as ECDSA keys and public key derivation is done by computing the offset $\omega := H(mpk, ch, ID)$, where $ID \in [2^{32}]$ and then rerandomizing mpk to pk_{ID} by computing $pk_{ID} := mpk + G \cdot \omega$. Here, G denotes the base point of an elliptic curve group of prime order p . A matching secret key can be derived via $sk_{ID} := msk + \omega \pmod{p}$.

Hot/Cold Wallets. A typical way of using deterministic wallets in practice is via the hot/cold wallet paradigm. With this approach, Alice maintains two wallets. The first wallet is referred to as the *cold wallet*. It keeps the master secret key msk as well as the chaincode. The cold wallet is usually implemented via some simple storage device that should be almost permanently disconnected from the internet, so as to minimize the risk of attack. The second wallet is the so-called *hot wallet*, which is permanently online and keeps the master public key as well as the chaincode. Using the deterministic key derivation procedures, the two wallets can independently derive matching keys to use for one-time transaction on the public ledger. In a bit more detail, Alice uses her hot wallet as a low-security spending wallet which, at any point in time, keeps only a small amount of currency. Whenever the funds stored on the hot wallet exceed a certain amount, Alice can use the public key derivation algorithm to derive a new public key pk_{ID} on her hot wallet and transfer the excess funds to pk_{ID} . Note that this requires no interaction with the cold wallet. At a later point in time, the cold wallet can come online for a brief moment and spend the funds from pk_{ID} , using a matching secret key sk_{ID} derived via the secret key derivation algorithm. As far as security goes, we would like to ensure two properties. First, *unlinkability* ensures that keys derived from the same master key pair are indistinguishable from random keys, given that the hot wallet has not leaked the chaincode to the attacker. Second, *unforgeability* ensures that even if the hot wallet leaks the chaincode (e.g., because it has been corrupted), signatures from derived keys should still remain unforgeable. While unlinkability is easy to achieve, unforgeability is a much more subtle issue in this setting, as the derived keys are all correlated once the

chaincode has been revealed to the attacker. Hence, the standard unforgeability property of the underlying signature scheme is no longer sufficient to ensure unforgeability of signatures under these derived keys.

1.2 Limitations of Existing Works

The work of Das et al. [15] was the first to provide a formal model to reason about the aforementioned security properties. It also showed how to achieve secure constructions in their proposed model from various different signature schemes used in practice, e.g., Schnorr, BLS, and ECDSA. Notably, the latter construction is very practical and can be integrated directly with the (unmodified) Bitcoin system. In spite of these achievements, their work makes no progress towards formally proving security properties for the BIP32 wallet standard that is widely used in many real-world systems. Let us discuss the reasons for this in a little more detail.

First, the construction of Das et al. uses a *multiplicative rerandomization* to derive keys, in which keys for identity ID are computed from $\omega = H(mpk, ch, ID)$ as $pk_{ID} := mpk \cdot \omega$, and $sk_{ID} := msk \cdot \omega \pmod{p}$. By comparison, as we saw above, BIP32 uses an *additive rerandomization*. Although this might look like a minor difference, we will see later that the proof technique and security guarantees achieved by the additive version differ significantly from the multiplicative one. Second, the work of Das et al. does not consider the hierarchical key derivation mechanism provided by BIP32. Hierarchical deterministic wallets allow for keys in the wallet to act simultaneously as signing keys *and* as parent (master) keys to derive new child keys in their own right. As a useful example, consider a company that wishes to delegate new signing key pairs to different entities within the company. Unfortunately, it cannot be guaranteed that all entities in the company store their keys securely and some of them might be leaked to the adversary over time. Such a strong adversary cannot be captured by the model and constructions of Das et al. Since many wallets that are used in practice follow the BIP32 standard, it is crucial to provide a formal analysis of the scheme *as is*, meaning without any modifications to it.

1.3 Our Contributions

In this work we address the above shortcomings and provide, for the first time, a formal analysis of the full BIP32 specification in the hot/cold wallet setting. An important implication of our work is that we can establish the exact security that is achieved by the current standard, which also leads us to propose a minor modification that can significantly improve security without any additional costs.

Rerandomizing ECDSA. We begin by recalling the notion of *unforgeability under honestly rerandomized keys (UFCMA-HRK)* introduced by Das et al. [15]. As this notion will serve as the basis of our wallet constructions, we review it in detail below. Compared to the standard notion of unforgeability under chosen message attacks (UFCMA), the adversary in the UFCMA-HRK game initially obtains a challenge public key pk and gets to query for rerandomizations of pk . The game returns the rerandomized public key $\tilde{pk}$ together with the (uniformly chosen) randomness ρ that was used in the rerandomization process. The exact way that the rerandomization is actually done depends on the scheme; we are mostly interested in the case where ECDSA keys are additively rerandomized as

$pk + G \cdot \rho$. The game then allows the adversary to query for signatures relative to any of the rerandomized public keys that it has previously obtained from the game. It is considered successful if it can return a forgery relative to any of the requested keys $\tilde{pk}$ on a message for which it has not previously asked for a signature under $\tilde{pk}$. As observed by Das et al., this security notion is a weakened version of *unforgeability under rerandomized keys* [20] in which the adversary can choose the random coins ρ itself and provide them to the game. In Section 3, we prove that ECDSA with additive rerandomization satisfies UFCMA-HRK as long as *each message is signed only once per key*. A first attempt is to naively follow the approach of Das et al. who showed that ECDSA with multiplicative rerandomization satisfies UFCMA-HRK (without any restrictions on the number of signatures per message). The main idea of Das et al.'s reduction from UFCMA-HRK to UFCMA (both with respect to the ECDSA scheme) is to rely on a *related key attack* (RKA) that is present in the multiplicatively rerandomized version of the ECDSA scheme. Concretely, the RKA allows to transform a signature (r, s) on message m_0 relative to a key pk_0 into a signature $(r, s/\rho)$ on message m_1 that is valid under the related key $pk_1 = pk_0 \cdot \rho$, where ρ satisfies $\rho = \frac{H(m_0)}{H(m_1)}$. This attack can be leveraged by the reduction to answer all signing queries in the UFCMA-HRK game. More precisely, using the RKA, it is possible to transform signatures obtained from the signing oracle in the UFCMA game into signatures relative to any of the rerandomized keys in the UFCMA-HRK game (via programming of the random oracle). Hence, we are immediately faced with the following obstacle: this RKA does not work if keys are additively rerandomized.

Extending to Additive Rerandomization. To overcome this issue with the existing reduction, we present a new RKA which works for additively rerandomized ECDSA. The attack works as follows: given a signature (r, s) on m_0 relative to pk_0 , (r, s) is also a valid signature relative to the public key $pk_1 = pk_0 + \rho \cdot G$ on message m_1 , given that $\rho = (H(m_0) - H(m_1))/r$. Rather surprisingly, considering ECDSA's huge popularity, we are not aware of this attack having been noticed previously. Using our new RKA, we are now able to (almost) make the simulation of signatures in Das et al.'s approach work. However, there is a further issue that comes from the structure of the additive RKA. Suppose that the reduction is directed to program the random oracle H on a message m so as to provide the attacker with a signature relative to a (rerandomized) public key $\tilde{pk}$ in the UFCMA-HRK game. The above RKA forces the reduction to program H on a value that depends on a *particular signature* (r, s) on m , which it obtains from the signing oracle in the underlying UFCMA game. Now, the only signature on m that the reduction can hand to the adversary under $\tilde{pk}$ is (r, s) . If the adversary requests another signature *on the same message* m , we are not able to reply with a fresh signature, as we can program H on m only a single time. For this reason, we have to restrict ourselves to one-per-message unforgeability. We emphasize, however, that this notion of security (one signature per-message) is sufficient in our setting, as transactions are identified by unique nonces in most cryptocurrencies (including Bitcoin) and hence never signed twice. An additional benefit of our new reduction (compared to [15]) is that it only requires the weaker assumption that the underlying ECDSA scheme is one signature per-message unforgeable in its

own right. This is worth noting, as the work of Fersch et al. shows that ECDSA achieves this property in the random oracle model [18] (albeit with a very large security loss). By comparison, the unrestricted security (i.e., UFCMA) of ECDSA remains only a conjecture in the plain random oracle model. Our reduction also removes the need for the random salt present in Das et al.'s construction. This is an important improvement, as it allows using BIP32 without Bitcoin's scripting language, which was required by the construction of Das et al. due to their use of the salt. Finally, we remark that our reduction (by comparison to Das et al.) is *non-tight* and loses a factor proportional to the total number of keys derived in the UFCMA-HRK game. We provide further discussion on this issue in the next section and in Section 3.

Hierarchical Wallets. To complete the analysis of BIP32, the second part of our work focuses on formal security properties when supporting hierarchies in deterministic wallet constructions (as is the case for BIP32). As already hinted, the core difficulty in this setting is that some of the wallet's keys may be given to untrustworthy users who may leak their cold wallet keys to the adversary. If this happens, it is important to ensure that the adversary does not gain information about secret keys further up in the hierarchy. It is easy to see that this property is not achieved if all keys are derived using the derivation algorithms described so far: if the adversary learns $sk_{ID} = msk + \rho \pmod{p}$, where ρ is computed as $\rho = H(mpk, ch, ID)$, then it can recover msk as $msk = sk_{ID} - \rho \pmod{p}$ and learn all cold wallet keys that were ever derived using msk . Because of this, BIP32 offers a second mode of deriving keys called *hardened key derivation*. Hardened keys are derived by changing the computation of the offset ρ above to $\rho = H(msk, ch, ID)$. Now, even when learning sk_{ID} , it is not possible for the adversary to recover msk . The downside of hardened key derivation is that the hot and cold wallet can no longer independently derive keys (as the hot wallet does not know msk). Thus, this mode of derivation is not intended for use in the hot/cold wallet paradigm, but simply to create keys with a higher degree of security. These keys can either be stored (efficiently) as part of the main wallet or handed to users in the system without any concern for other cold wallet keys. In Section 4, we state the syntactical definition and correctness properties of a hierarchical deterministic wallet. We then introduce a security model that supports both types of key derivations (hardened and non-hardened), as well as secret key leakage of hardened keys. We refer to this notion of security as WUFCMA. In Section 5, we provide a generic construction HDWal that transforms a signature scheme satisfying UFCMA-HRK into a hierarchical deterministic wallet with WUFCMA security.¹ In this way, we are able to complete the analysis of BIP32 by instantiating HDWal with ECDSA using additive rerandomization.

On the Tightness of Our Construction. A particular focus of our work is to analyze the tightness and concrete security achieved by our constructions, most notably BIP32. We have already mentioned that our reduction from UFCMA-HRK to UFCMA of the ECDSA scheme with additive rerandomization is non-tight. More precisely, it loses a factor proportional to the number of keys derived by the adversary in the UFCMA-HRK game. Thus, our goal is to at least

¹In case the underlying signature scheme has the one signature per message restriction, then the resulting wallet scheme also does.

achieve the best possible tightness of our generic transform HDWal. To this end, let us first consider the possible options for potential security losses. From worst to best (excluding a tight reduction), the options are:

- Loss in the number of random oracle queries.
- Loss in the number of keys derived in the wallet (hardened or non-hardened).
- Loss in the number of signing oracle queries (assuming keys are used only once).
- Loss in the number of hardened keys leaked to the adversary.

The first three possibilities are quite catastrophic as the number of random oracle queries, signing oracle queries, or keys derived in practice could be quite high. On the other hand, we expect the number of *leaked keys* to be only a small portion of all the keys in a given wallet (we use 1% as an estimate in our calculations). We are able to prove that HDWal indeed achieves a multiplicative security loss proportional to only the hardened keys leaked to the adversary over the course of the lifetime of the wallet. Furthermore, we show that any *generic transform* from UFCMA-RK (a stronger notion than what is used in our construction) to WUFCMA *must lose at least this factor*. Hence, our construction HDWal achieves the *best possible parameters*. To prove our results, we adapt the reduction/metareduction techniques introduced by Coron in his seminal work [13]. Given that his results deal with the tightness of unique signatures (which is very different from our setting), this requires careful insight into his technique in order to adapt it to our model.

Concrete Security Parameters. We conclude by giving a discussion of the concrete security levels achieved by BIP32 and the multiplicative ECDSA scheme of Das et al., when plugged into HDWal. We find that BIP32 gives roughly 94 bits of security according to our theorems and conservative choices of parameters. We find that by comparison, the multiplicative version of Das et al. gives 114 bits of security with a similarly efficient scheme. (We remark that using the techniques introduced in our paper, we can also remove the salt in the multiplicatively rerandomizable ECDSA version of Das et al.). Given these insights, we strongly recommend that the Bitcoin community switch rerandomizations in BIP32 from additive to multiplicative, in particular since these changes essentially come for free.

1.4 Related Work

The most relevant previous work for us is by Das et al. [15] as mentioned previously. However, there have been other works which try to formalize cryptographic wallets. The work of Gutoski and Stebila [22] proposes an alternative construction for hierarchical wallets where up to d session keys can leak without the master secret key being compromised under the one-more discrete-log assumption. However, their security model is weaker than our model (or the security model of Das et al. on which we base our work). More precisely, in their model, the adversary cannot query the game for signatures under uncompromised wallet keys. Furthermore, instead of the traditional security model where the adversary wins if she can forge a signature, the adversary's goal in their security definition is to extract the master secret/public key pair. Another more recent work is by Luzio et al. [27] where the authors design

a new hierarchical wallet scheme by using (deterministic) hierarchical key assignment schemes [6]. Unfortunately, their solution is not compatible with cryptocurrencies such as Bitcoin since their solution requires a more sophisticated (signature) verification algorithm, where a certificate associated with the user needs to be verified along with the signature.

Turuani et al. [32] analyzed the Bitcoin Electrum wallet using automated verification in the Dolev-Yao model. However, many automated verification models only consider “idealized” building blocks, i.e., cryptographic building blocks that are perfectly secure. Consequently, this type of analysis excludes weaknesses such as related key attacks, which are of fundamental relevance in the setting of deterministic wallets.

Another line of work has considered the security of hardware wallets [5, 28] and implementation bugs in wallets (such as weak randomness) [10, 11, 14]. Additionally, there have been several works with focus on the use of threshold ECDSA signatures [16, 21, 25, 26] and multi-signatures [9] in (and outside of) wallet systems.

In a recent work, Alkadri et al. [4] have shown how to realize deterministic wallets that are post-quantum secure. To this end, they suitably adapt the model and techniques of Das et al. by considering an adversary with quantum computing power.

The concept of rerandomizable signature schemes was first introduced by Fleischhacker et al. [20] and later used by [4, 15] for their wallet schemes. In addition, related key attacks have been studied for signature schemes such as Schnorr [29] in many previous works [19, 24, 34]. For ECDSA, Das et al. leveraged related key attacks to achieve a multiplicatively rerandomizable ECDSA scheme which they prove secure w.r.t. the security notion of unforgeability under honestly rerandomizable keys. Finally, Fersch et al. [17] provided the first security analysis of ECDSA in an idealized model.

2 PRELIMINARIES

Notation. We use the notation $s \xleftarrow{\$} H$ to denote the uniform sampling of a variable s from the set H . For an integer l , $[l]$ denotes the set of integers $\{1, \dots, l\}$. We use upper case letters to denote algorithms. For an algorithm A , we write $y \xleftarrow{\$} A(x)$ to denote the execution of a randomized algorithm A on input x that outputs y . We write $y \leftarrow B(x; \rho)$ to denote the execution of an algorithm B that, on input x and randomness ρ , outputs y . Note that in this notation, B is *deterministic*. We use the notation $y \in A(x)$ to denote that y is in the set of possible outputs of A on input x .

In order to simplify our notation and definitions, we assume that public parameters par have been securely generated and can be used throughout the paper as input to algorithms. We generally assume that, initially, boolean variables are set to false, integers are set to 0, lists are set to $\emptyset$, and undefined entries of lists are set to $\perp$. For strings $a, b \in \{0, 1\}^*$, we write $a = (b, \cdot)$ if b is a prefix of a and likewise, we write $a \neq (b, \cdot)$ if a is not prefixed by b . We denote by κ the security parameter throughout the paper.

We use standard code-based security games [30]. A *game* G is an interactive probability experiment between an *adversary* $\mathcal{A}$ and an (implicit) *challenger* which provides answers to oracle queries posed by $\mathcal{A}$. The output of G when interacting with adversary $\mathcal{A}$ is denoted as $G^{\mathcal{A}}$. Finally, the randomness in any probability term

of the form $\Pr[G^{\mathcal{A}} = 1]$ is assumed to be over all the random coins in game G .

2.1 Signature Schemes

We now recall the definition of signature schemes and that of signature schemes with perfectly rerandomizable keys from [15].

Definition 2.1 (Signature Scheme). A *signature scheme* is a tuple of algorithms $\text{Sig} = (\text{Sig.Gen}, \text{Sig.Sign}, \text{Sig.Verify})$ which are defined as follows:

- $\text{Sig.Gen}(\text{par})$: The randomized *key generation* algorithm Sig.Gen takes as input public parameters par and outputs a public/secret key pair (pk, sk) .
- $\text{Sig.Sign}(\text{sk}, m)$: The (possibly) randomized *signing* algorithm Sig.Sign takes as input a secret key sk and a message m and outputs a signature σ .
- $\text{Sig.Verify}(m, \text{pk}, \sigma)$: The deterministic *verification* algorithm Sig.Verify takes as input a public key pk , a signature σ , and a message m . It outputs either 1 (accept) or 0 (reject).

A signature scheme Sig is *correct* if the following holds: For all $(\text{pk}, \text{sk}) \in \text{Sig.Gen}(\text{par})$ and all $m \in \{0, 1\}^*$ we have that

$$\Pr_{\sigma \leftarrow \text{Sig.Sign}(\text{sk}, m)} [\text{Sig.Verify}(\text{pk}, \sigma, m) = 1] = 1.$$

Definition 2.2 (Signature Scheme with Perfectly Rerandomizable Keys). A *signature scheme with perfectly rerandomizable keys* is a tuple of algorithms $\text{RSig} = (\text{RSig.Gen}, \text{RSig.Sign}, \text{RSig.Verify}, \text{RSig.RandSK}, \text{RSig.RandPK})$ where $(\text{RSig.Gen}, \text{RSig.Sign}, \text{RSig.Verify})$ are the standard algorithms of a signature scheme. Moreover, we assume that the public parameters par define a randomness space $\mathcal{R} := \mathcal{R}(\text{par})$. Then the algorithms RSig.RandSK and RSig.RandPK are defined as follows:

- $\text{RSig.RandSK}(\text{sk}; \rho)$: The deterministic *secret key rerandomization algorithm* RSig.RandSK takes as input a secret key sk and randomness $\rho \in \mathcal{R}$ and outputs a rerandomized secret key sk' .
- $\text{RSig.RandPK}(\text{pk}; \rho)$: The deterministic *public key rerandomization algorithm* RSig.RandPK takes as input a public key pk and randomness $\rho \in \mathcal{R}$ and outputs a rerandomized public key pk' .

We make the convention that for the empty string ϵ , we have that $\text{RSig.RandPK}(\text{pk}; \epsilon) = \text{pk}$ and $\text{RSig.RandSK}(\text{sk}; \epsilon) = \text{sk}$.

We further require:

- (1) *(Perfect) rerandomizability of keys*: For all $(\text{sk}, \text{pk}) \in \text{RSig.Gen}(\text{par})$ and $\rho \leftarrow \mathcal{R}$, the distributions of (sk', pk') and $(\text{sk}'', \text{pk}'')$ are identical, where:

$$(\text{sk}', \text{pk}') \leftarrow (\text{RSig.RandSK}(\text{sk}; \rho), \text{RSig.RandPK}(\text{pk}; \rho)),$$

$$(\text{sk}'', \text{pk}'') \leftarrow \text{RSig.Gen}(\text{par}).$$
- (2) *Correctness under rerandomized keys*: For all $(\text{sk}, \text{pk}) \in \text{RSig.Gen}(\text{par})$, for all $\rho \in \mathcal{R}$, and for all $m \in \{0, 1\}^*$, the rerandomized keys $\text{sk}' \leftarrow \text{RSig.RandSK}(\text{sk}; \rho)$ and $\text{pk}' \leftarrow \text{RSig.RandPK}(\text{pk}; \rho)$ satisfy:

$$\Pr_{\sigma \leftarrow \text{RSig.Sign}(\text{sk}', m)} [\text{RSig.Verify}(\text{pk}', \sigma, m) = 1] = 1.$$

Security notion uf-cma1. In this work, we use the security notion of *one-per message existential unforgeability under chosen message*

attacks (uf-cma1) [18] which is a slightly weaker variant of the standard notion of existential unforgeability under chosen message attacks (uf-cma) security. In contrast to standard uf-cma, in uf-cma1, the adversary is restricted to querying the signing oracle at most once for each message. We formalize the uf-cma1 notion for a signature scheme Sig in the form of a game $\text{uf-cma1}_{\text{Sig}}$ as follows.

Game $\text{uf-cma1}_{\text{Sig}}$:

- **Setup Phase:** The challenger initiates a list as $\text{SigList} \leftarrow \{\epsilon\}$ for storing messages and samples a pair of keys $(\text{pk}, \text{sk}) \xleftarrow{\$} \text{Sig.Gen}(\text{par})$. Then, $\mathcal{A}$ is run on input pk .
- **Online Phase:** $\mathcal{A}$ is given access to a signing oracle Sign which works as follows. On input a message m , if m was queried in a previous Sign query, i.e., if $m \in \text{SigList}$, then $\perp$ is returned. Otherwise, Sign computes a signature on message m as $\sigma \xleftarrow{\$} \text{Sig.Sign}(\text{sk}, m)$. The message m is stored in the SigList and the signature σ is returned as the answer.
- **Output Phase:** Finally, $\mathcal{A}$ wins the game if it can provide a forgery σ^* on a message m^* , where (1) m^* is fresh, i.e., $m^* \notin \text{SigList}$ and (2) σ^* is a valid forgery, i.e., $\text{Sig.Verify}(\text{pk}, \sigma^*, m^*) = 1$.

For an algorithm $\mathcal{A}$ we define $\mathcal{A}$'s advantage in the game $\text{uf-cma1}_{\text{Sig}}$ as $\text{Adv}_{\text{uf-cma1}_{\text{Sig}}}^{\mathcal{A}} = \Pr[\text{uf-cma1}_{\text{Sig}}^{\mathcal{A}} = 1]$.

Security notion uf-cma-hrk1. For signature schemes with perfectly rerandomizable keys, we introduce the notion of *one-per message existential unforgeability under honestly rerandomizable keys* (uf-cma-hrk1), which restricts the security notion of *existential unforgeability under honestly rerandomizable keys* (uf-cma-hrk) as introduced by Das et al. [15]. In this security notion, the signing oracle cannot only return signatures under sk , but it can also return signatures that were produced with keys that represent *honest* rerandomizations of sk . The term *honest* indicates that the randomness for the rerandomization is chosen uniformly at random from $\mathcal{R}$ (by the game itself). Our security notion of uf-cma-hrk1 restricts the notion of uf-cma-hrk in the sense that the signing oracle returns at most one signature for each randomness/message pair (ρ, m) . We formally model the notion of uf-cma-hrk1 for a rerandomizable signature scheme RSig in the form of a game $\text{uf-cma-hrk1}_{\text{RSig}}$ as follows.

Game $\text{uf-cma-hrk1}_{\text{RSig}}$:

- **Setup Phase:** The challenger initializes two lists as $\text{SigList} \leftarrow \{\epsilon\}$ and $\text{RList} \leftarrow \{\epsilon\}$ and samples a pair of keys $(\text{pk}, \text{sk}) \xleftarrow{\$} \text{RSig.Gen}(\text{par})$. Then $\mathcal{A}$ is run on input pk .
- **Online Phase:**
 - $\mathcal{A}$ is given access to an oracle Rand , which, upon a query, samples a fresh random value from $\mathcal{R}$ as $\rho \xleftarrow{\$} \mathcal{R}$, stores ρ in the list RList , and returns ρ .
 - $\mathcal{A}$ is given access to a signing oracle RSign which works as follows. On input a message m and a randomness ρ , if ρ was not obtained via a prior Rand query (i.e., $\rho \notin \text{RList}$), then return $\perp$. Otherwise, derive a pair of keys rerandomized with the randomness ρ , as $\text{sk}' \leftarrow \text{RSig.SKDer}(\text{sk}; \rho)$ and $\text{pk}' \leftarrow \text{RSig.PKDer}(\text{pk}; \rho)$. If $(\text{pk}', m) \in \text{SigList}$ then return $\perp$. Otherwise, a signature is derived on message m under the secret key sk' as $\sigma \leftarrow \text{RSig.Sign}(\text{sk}', m)$. The tuple (pk', m)

is stored in the SigList and the signature σ is returned as the answer.

- **Output Phase:** $\mathcal{A}$ wins if it returns a forgery σ^* together with a message m^* and a public key $pk^* \leftarrow \text{RSig.PKDer}(pk; \rho^*)^2$, s.t. following holds: (1) the randomness ρ^* has been derived via a Rand query, i.e., $\rho^* \in \text{RList}$, (2) (m^*, ρ^*) is fresh, i.e., $(pk^*, m^*) \notin \text{SigList}$, and (3) σ^* is a valid forgery, i.e., $\text{RSig.Verify}(pk^*, \sigma^*, m^*) = 1$.

For an algorithm $\mathcal{A}$ we define $\mathcal{A}$'s advantage in game **uf-cma-hrk1**_{RSig} as $\text{Adv}_{\text{uf-cma-hrk1}_{\text{RSig}}}^{\mathcal{A}} = \Pr[\text{uf-cma-hrk1}_{\text{RSig}}^{\mathcal{A}} = 1]$.

Other than only allowing the adversary to ask for at most one signature per message, our definition deviates from the one presented in [15] by storing the tuples (pk', m) in the list SigList instead of just storing m . This change allows an adversary in the **uf-cma-hrk1** game to query a signature for the same message but under different public keys.

3 SECURITY ANALYSIS OF ADDITIVELY RERANDOMIZABLE ECDSA

In the following discussion, let $\mathbb{E}(\text{par})$ denote an elliptic curve with base point G and prime order p . Furthermore, assume hash functions $H_0: \{0, 1\}^* \rightarrow \mathbb{Z}_p$, $H_1: \{0, 1\}^* \rightarrow \mathbb{Z}_p$ (modeled as random oracles). In this section, we present a signature scheme with rerandomizable keys $\text{REC}[H_1]$ based on the standard ECDSA scheme which we denote by $\text{EC}[H_0]$ (cf. Figure 1). $\text{REC}[H_1]$, as illustrated in Figure 7, works in a similar way as $\text{EC}[H_0]$ with two main differences. (1) It is extended by two algorithms RandSK and RandPK for the key rerandomization and (2) it is designed for key-prefixed messages. First, the two algorithms RandSK and RandPK randomize a key pair by *adding* a random value to each key. This is in contrast to the signature scheme with *multiplicatively* rerandomizable keys based on ECDSA as presented by Das et al. [15], where the rerandomization algorithms multiply a random value to each key. Second, $\text{REC}[H_1]$ is designed for key-prefixed messages, i.e., upon executing $\text{REC}[H_1].\text{Sign}(sk, m)$ for a secret key sk and a message m , the message is first extended to a *key-prefixed message* $pm \leftarrow (pk, m)$ where pk represents the public key corresponding to sk . Then the prefixed message pm is signed under sk .

We prove that $\text{REC}[H_1]$ satisfies **uf-cma-hrk1** security by providing a reduction from the **uf-cma1** security of the standard ECDSA scheme $\text{EC}[H_0]$. An integral part of the reduction is the observation that there exists a so-called “related key attack” (RKA) in the scheme $\text{EC}[H_0]$. An RKA allows to transform a signature that is valid under a public key pk_0 into a signature that is valid under another public key pk_1 given there exists a specific relation between pk_1 and pk_0 . The RKA in $\text{EC}[H_0]$ allows to use a signature σ that is valid under a public key pk_0 as a valid signature under a public key pk_1 in case pk_1 and pk_0 are related as $pk_1 = pk_0 + \rho \cdot G$, where ρ must satisfy $\rho = \frac{H_0(m_0) - H_1(m_1)}{r}$. We formally describe this related key attack in the following Lemma.

²For simplicity, we tacitly assume that pk^* identifies ρ^* . This can easily be achieved using appropriate bookkeeping.

Algorithm $\text{EC}[H_0].\text{Gen}(\text{par})$	Algorithm $\text{EC}[H_0].\text{Verify}(pk = X, \sigma, m)$
00 $x \xleftarrow{\$} \mathbb{Z}_p$	15 Parse $(r, s) \leftarrow \sigma$
01 $X \leftarrow x \cdot G$	16 If $(r, s) \notin \mathbb{Z}_p$
02 $sk \leftarrow x$	17 Return 0
03 $pk \leftarrow X$	18 $w \leftarrow s^{-1} \bmod p$
04 Return (pk, sk)	19 $z \leftarrow H_0(m)$
Algorithm $\text{EC}[H_0].\text{Sign}(sk = x, m)$	20 $u_1 \leftarrow zw \bmod p$
05 $z \leftarrow H_0(m)$	21 $u_2 \leftarrow rw \bmod p$
06 $t \xleftarrow{\$} \mathbb{Z}_p$	22 $(e_x, e_y) \leftarrow u_1 \cdot G + u_2 \cdot X$
07 $(e_x, e_y) \leftarrow t \cdot G$	23 If $(e_x, e_y) = (0, 0)$
08 $r \leftarrow e_x \bmod p$	24 Return 0
09 If $r = 0 \bmod p$	25 Return $r = e_x \bmod p$
10 Goto Step 06	
11 $s \leftarrow t^{-1}(z + rx) \bmod p$	
12 If $s = 0 \bmod p$	
13 Goto Step 06	
14 Return $\sigma := (r, s)$	

Figure 1: $\text{EC}[H_0] = (\text{EC}[H_0].\text{Gen}, \text{EC}[H_0].\text{Sign}, \text{EC}[H_0].\text{Verify})$: ECDSA signature scheme over to elliptic curve $\mathbb{E}$ using hash function $H_0: \{0, 1\}^* \rightarrow \mathbb{Z}_p$.

Algorithm $\text{REC}[H_1].\text{Sign}(sk, m)$	Algorithm $\text{REC}[H_1].\text{RandSK}(sk; \rho)$
00 $pm \leftarrow (pk, m)$	00 $sk' \leftarrow (sk + \rho) \bmod p$
01 $\sigma \leftarrow \text{EC}[H_1].\text{Sign}(sk, pm)$	01 Return sk'
02 Return σ	
Algorithm $\text{REC}[H_1].\text{Verify}(pk, \sigma, m)$	Algorithm $\text{REC}[H_1].\text{RandPK}(pk; \rho)$
03 $pm \leftarrow (pk, m)$	02 $pk' \leftarrow (pk + \rho \cdot G)$
04 Return $\text{EC}[H_1].\text{Verify}(pk, \sigma, pm)$	03 Return pk'

Figure 2: Key-prefixed version of the ECDSA signature scheme with perfectly rerandomizable keys $\text{REC}[H_1] := (\text{REC}[H_1].\text{Gen} = \text{EC}[H_1].\text{Gen}, \text{REC}[H_1].\text{Sign}, \text{REC}[H_1].\text{Verify}, \text{REC}[H_1].\text{RandSK}, \text{REC}[H_1].\text{RandPK})$ based on the ECDSA signature scheme $\text{EC}[H_1]$. Above $H_1: \{0, 1\}^* \rightarrow \mathbb{Z}_p$ denotes a hash function.

Lemma 3.1 Let $H_0, H_1: \{0, 1\}^* \rightarrow \mathbb{Z}_p$ be hash functions (modeled as random oracles). Suppose that $\sigma = (r, s)$ is a valid signature on message $m_0 \in \{0, 1\}^*$ w.r.t. $\text{EC}[H_0]$ and public key pk_0 , i.e., $\text{EC}[H_0].\text{Verify}(pk_0, \sigma, m_0) = 1$. Furthermore, let $\rho = \frac{H_0(m_0) - H_1(m_1)}{r} \bmod p$. Then σ is also a valid signature on message $m_1 \in \{0, 1\}^*$ w.r.t. $\text{EC}[H_1]$ and public key $pk_1 = pk_0 + \rho \cdot G$, i.e., $\text{EC}[H_1].\text{Verify}(pk_1, \sigma, m_1) = 1$.

PROOF OF LEMMA 3.1. We have to show that $\text{EC}[H_1].\text{Verify}(pk_1, \sigma, m_1) = 1$ for $pk_1 = pk_0 + \rho \cdot G$ and $\rho = \frac{H_0(m_0) - H_1(m_1)}{r} \bmod p$. Note that $\sigma = (r, s)$, where $s = t^{-1}(H_0(m_0) + rsk_0) \bmod p$ and r represents the x -coordinate of the elliptic curve point $t \cdot G$ for $t \xleftarrow{\$} \mathbb{Z}_p$. As shown in Figure 1, $\text{EC}[H_1].\text{Verify}(pk_1, \sigma, m_1)$ computes the following:

$$\begin{aligned}
& u_1 \cdot G + u_2 \cdot pk_1 \\
&= H_1(m_1) \cdot s^{-1} \cdot G + r \cdot s^{-1} \cdot \left(pk_0 + \frac{H_0(m_0) - H_1(m_1)}{r} \cdot G \right) \\
&= s^{-1} \cdot G (H_1(m_1) + r \cdot sk_0 + H_0(m_0) - H_1(m_1)) \\
&= s^{-1} \cdot G (r \cdot sk_0 + H_0(m_0)) \\
&= t \cdot (H_0(m_0) + rsk_0)^{-1} \cdot (H_0(m_0) + rsk_0) \cdot G = t \cdot G
\end{aligned}$$

Since the x -coordinate of $t \cdot G$ equals $r \pmod{p}$, it holds that $\text{EC}[H_1].\text{Verify}(pk_1, \sigma, m_1) = 1$. ■

The RKA from Lemma 3.1 can be extended to an RKA between the schemes $\text{EC}[H_0]$ and $\text{REC}[H_1]$ such that a valid signature under pk_0 for a *prefixed* message $pm \leftarrow (pk_1, m)$ in $\text{EC}[H_0]$ is also valid in $\text{REC}[H_1]$ under pk_1 for message m . This RKA allows to transfer a valid signature from $\text{EC}[H_0]$ to a valid signature in $\text{REC}[H_1]$ and vice versa in case pk_0 and pk_1 satisfy the relation from Lemma 3.1. We formally present this RKA in the following Lemma.

Lemma 3.2 *Let $H_0, H_1: \{0, 1\}^* \rightarrow \mathbb{Z}_p$ be hash functions (modeled as random oracles). Let $m \in \{0, 1\}^*$ and suppose that $\sigma = (r, s)$ is a valid signature on message $pm \leftarrow (pk_1, m)$ w.r.t. $\text{EC}[H_0]$ and public key pk_0 , i.e., $\text{EC}[H_0].\text{Verify}(pk_0, \sigma, pm) = 1$. Furthermore, suppose that $pk_1 = pk_0 + \rho \cdot G$ where $\rho = \frac{H_0(pm) - H_1(pm)}{r} \pmod{p}$. Then σ is also a valid signature on message m w.r.t. $\text{REC}[H_1]$ and public key pk_1 , i.e., $\text{REC}[H_1].\text{Verify}(pk_1, \sigma, m) = 1$.*

PROOF OF LEMMA 3.2. We have to show that $\text{REC}[H_1].\text{Verify}(pk_1, \sigma, m) = 1$ for $pk_1 = pk_0 + \rho \cdot G$ and $\rho = \frac{H_0(pm) - H_1(pm)}{r} \pmod{p}$, where $pm \leftarrow (pk_1, m)$. Note that $\sigma = (r, s)$, where $s = t^{-1}(H_0(pm) + rsk_0) \pmod{p}$ and r represents the x -coordinate of the elliptic curve point $t \cdot G$ for $t \xleftarrow{\$} \mathbb{Z}_p$. As shown in figure 7, $\text{REC}[H_1].\text{Verify}(pk_1, \sigma, m)$ first computes the prefixed message $pm \leftarrow (pk_1, m)$ and then runs $\text{EC}[H_1].\text{Verify}(pk_1, \sigma, pm)$. The rest follows from the proof of Lemma 3.1 with $m_0 = m_1 = pm$. ■

3.1 Security analysis of REC

In this section, we analyze the one-per message unforgeability of the honestly rerandomizable signature scheme, or in short the **uf-cma-hrk1** security of the scheme $\text{REC}[H_1]$. We prove the following theorem.

Theorem 3.3 *Let $H_0, H_1: \{0, 1\}^* \rightarrow \mathbb{Z}_p$ be hash functions (modeled as random oracles). Let $\mathcal{A}$ be an algorithm that plays in the game **uf-cma-hrk1** $_{\text{REC}[H_1]}$. Then there exists an algorithm $\mathcal{C}$ running in roughly the same time as $\mathcal{A}$, such that*

$$\text{Adv}_{\text{uf-cma1}_{\text{EC}[H_0]}}^{\mathcal{C}} \geq \left(\text{Adv}_{\text{uf-cma-hrk1}_{\text{REC}[H_1]}}^{\mathcal{A}} - \frac{q_{H_1}^2}{p} \right) \cdot \frac{1}{q}$$

where q_{H_1} and q are the number of random oracle queries and Rand queries, respectively, that $\mathcal{A}$ makes.

Due to space limitations, we present the full proof of Theorem 3.3 in Appendix A. We now give some intuition on how we overcome the main difficulties in our simulation. At a high level, the idea is to reduce the **uf-cma-hrk1** security of the additively rerandomizable ECDSA construction $\text{REC}[H_1]$ from the **uf-cma1** security of ECDSA construction $\text{EC}[H_0]$. Therefore, the proof essentially consists of building a reduction $\mathcal{C}$ trying to come up with a valid forgery to win the **uf-cma1** $_{\text{EC}[H_0]}$ game, by simulating the **uf-cma-hrk1** $_{\text{REC}[H_1]}$ game to adversary $\mathcal{A}$ using the RKA from Lemma 3.2. In the **uf-cma1** $_{\text{EC}[H_0]}$ game, $\mathcal{C}$ obtains a public key pk_C from its challenger. It can query an oracle **Sign** to get signatures w.r.t. pk_C . $\mathcal{C}$ also has access to a random oracle H_0 . $\mathcal{C}$'s goal is to somehow embed its public key pk_C in one of the rerandomized public keys pk^* under which $\mathcal{A}$ eventually returns a forgery (pk^*, σ^*, m^*) . The hope is that $\mathcal{C}$ can use (pk^*, σ^*, m^*) to win its own game **uf-cma1** $_{\text{EC}[H_0]}$.

In more detail, $\mathcal{C}$'s strategy works as follows. Instead of directly using pk_C , $\mathcal{C}$ generates the challenge public key for $\mathcal{A}$ by additively shifting pk_C with a freshly sampled $\tilde{\rho} \xleftarrow{\$} \mathcal{R}$, i.e., $pk \leftarrow pk_C - \tilde{\rho} \cdot G$. When $\mathcal{A}$ asks for a signature under a key $pk' = pk + \rho \cdot G$, $\mathcal{C}$ can simulate such signatures by querying its **Sign** oracle and employing the RKA from Lemma 3.2. This is because, to the adversary $\mathcal{A}$, pk' looks like a rerandomization of pk , while in fact, it is derived from pk_C as $pk' = pk + \rho \cdot G = (pk_C - \tilde{\rho} \cdot G) + \rho \cdot G$. To make this simulation work, the random oracle H_1 must be carefully programmed by $\mathcal{C}$ such that the relation between ρ , H_0 and H_1 satisfies $H_1(m) = H_0(m) - r \cdot \rho \pmod{p}$ (according to Lemma 3.2), where $(r, s) := \sigma$ is the signature³. Note that, due to the programming of the random oracle, the first simulated signature for every message and randomness pair (m, ρ) fully determines $H_1(m)$. Hence, the simulated signing oracle in **uf-cma-hrk1** $_{\text{REC}[H_1]}$ can be queried at most once on every input pair (m, ρ) . $\mathcal{C}$'s strategy to win **uf-cma1** $_{\text{EC}[H_0]}$ is to embed $\tilde{\rho}$ at random as an answer to one of the Rand queries in **uf-cma-hrk1** $_{\text{REC}[H_1]}$. For signing queries w.r.t. pk , $\mathcal{C}$ does not reprogram H_1 ; instead, it uses H_0 and signatures obtained from the signing oracle in **uf-cma1** $_{\text{EC}[H_0]}$ directly. If $\mathcal{A}$ returns a valid forgery σ^* w.r.t. to $pk^* = pk + \tilde{\rho} \cdot G$, then $\mathcal{C}$ can simply use this forgery to win the **uf-cma1** $_{\text{EC}[H_0]}$ game. This is because $pk^* = pk + \tilde{\rho} \cdot G = pk_C - \tilde{\rho} \cdot G + \tilde{\rho} \cdot G = pk_C$. Note that pk^* is the *only* key for which the forgery σ^* is valid in game **uf-cma1** $_{\text{EC}[H_0]}$. For any other key pk' , the simulation of the signing oracle in **uf-cma-hrk1** $_{\text{REC}[H_1]}$ requires to reprogram H_1 on any message that is prefixed with pk' . Since this involves a signing query on that very message to the signing oracle in **uf-cma1** $_{\text{EC}[H_0]}$, the forgery would no longer be fresh in the latter game. This guessing on $\mathcal{C}$'s part is also the reason that our reduction is not tight.

4 A MODEL FOR HIERARCHICAL DETERMINISTIC WALLETS

In this section, we introduce a formal model for hierarchical deterministic wallets. This model closely reflects the BIP32 specification [33] with only minor differences which we list in Section 6. At a high level, a hierarchical deterministic wallet scheme can be visualized as a tree, where every node in the tree corresponds to a wallet. As is usual in a tree structure, the scheme originates from a root node, which contains a pair of master keys - a master public key mpk and a master secret key msk as well as a seed $ch_{0,0}$ which we will refer to as chaincode from now on. We say that the root node is located at level 0 of the tree. The root can create a child node at level 1 and position t by deriving a new key pair $(pk_{1,t}, sk_{1,t})$ and a chaincode $ch_{1,t}$ from its master keys and chaincode $ch_{0,0}$. This child node represents a new wallet that is initiated with the key pair $(pk_{1,t}, sk_{1,t})$ and chaincode $ch_{1,t}$ and using these values it can in turn create a child node for level 2. This child creation process can continue recursively. Note, however, that a node at level i can only create children for the *immediate* lower level, i.e., for level $i + 1$.

In our model, we distinguish between two different kinds of nodes, namely *non-hardened* and *hardened* nodes. Non-hardened

³ An important aspect of this simulation is that $\mathcal{C}$ can program H_1 whenever it observes a query m to H_1 that is prefixed with a previously rerandomized key. In particular, this can be done *before* m is ever queried to the signing oracle in **uf-cma-hrk1** $_{\text{REC}[H_1]}$.

nodes are, in essence, the nodes as discussed above, i.e., nodes that can be used for child creation at the next lower level. We assume that the public key and the chaincode of a non-hardened node can be corrupted by an adversary, whereas the secret key remains protected. One might think of non-hardened nodes as wallets in the hot/cold wallet setting, where the hot wallet stores the public key, the cold wallet stores the secret key and the chaincode is provided to both wallets. While the hot wallet is permanently online and thereby vulnerable to attacks, the cold wallet stays offline for the majority of the time and is therefore protected against attacks. To create a non-hardened child node at level i and at position t , its parent must generate the child node's key pair $(pk_{i,t}, sk_{i,t})$ and chaincode $ch_{i,t}$. We model the derivation of these values in such a way that the derivation process of $sk_{i,t}$ involves the parent's secret key, while the derivation of $pk_{i,t}$ and $ch_{i,t}$ requires only the parent's public key and chaincode (i.e., it is independent of the parent's secret key).

Hardened nodes, on the other hand, represent the leaves of the tree, i.e., we do not consider any child derivation from hardened nodes⁴. However, in comparison to non-hardened nodes we allow secret key leakage, along with public key and chaincode leakage for hardened nodes. That is, we consider full corruption of hardened nodes. Our security goal is that the secret key leakage of a hardened node does not affect the security of any other node in the tree. As opposed to non-hardened nodes, the creation process of a hardened child node requires the secret key of the parent node, i.e., even for the derivation of the child's public key and chaincode. The tree structure of a hierarchical deterministic wallet scheme, containing hardened as well as non-hardened nodes can be found in Figure 3.

While hardened nodes clearly exhibit stronger security guarantees than non-hardened nodes, the advantage of non-hardened nodes lies in the child creation process. We will illustrate this advantage in the following example. In a company there might be trusted and untrusted employees. Trusted employees operate a non-hardened node, as they are trusted to properly protect their secret key, e.g., by storing it in a cold wallet. On the other hand, untrusted employees have to operate a hardened node as they might leak their secret key or simply get compromised. Assume a trusted employee maintains a non-hardened node with key pair $(pk_{i,t}, sk_{i,t})$ and chaincode $ch_{i,t}$. Further assume that the node is operated in a hot/cold wallet setting, i.e., the tuple $(sk_{i,t}, ch_{i,t})$ is stored in a cold wallet and the tuple $(pk_{i,t}, ch_{i,t})$ is stored in a hot wallet. If the employee wishes to receive payments to different public addresses, it can simply generate these addresses by deriving non-hardened child public keys using only the information stored in its hot wallet. In particular, the cold wallet can remain offline during this process. Only when the employee wants to spend the coins it received, it has to use $sk_{i,t}$ from the cold wallet to generate the secret keys corresponding to the public addresses it generated earlier.

Another example for the usefulness of non-hardened nodes is the following. Consider a company A that operates a non-hardened node with key pair $(pk_{i,t}, sk_{i,t})$ and chaincode $ch_{i,t}$ only to receive payments from a company B. In this case, company A can simply share $pk_{i,t}$ and $ch_{i,t}$ with company B, which can then by itself

generate non-hardened child public keys and make the payments to those addresses. Note that in this case, company A does not have to be involved in the payment process at all.

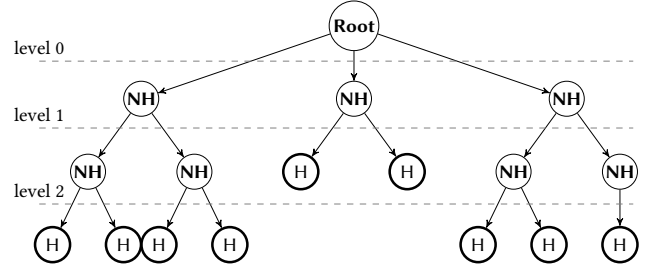


Figure 3: Tree structure of a hierarchical deterministic wallet scheme. Hardened nodes are denoted by H while non-hardened nodes are denoted by NH.

Flat Vs Hierarchical Deterministic Wallets. Let us now briefly discuss the main difference between the model for hierarchical deterministic wallets and the setting originally analyzed by Das et al [15] which we denote as *the flat model*. The key derivation process in the flat model works in the same way as the non-hardened key derivation in the hierarchical model with the difference that the flat model allows to derive keys only directly from the master key pair. Hardened nodes are not considered in the flat model. Therefore, the flat model basically represents a hierarchical wallet structure with non-hardened leaf nodes at level 1 (see Figure 4). Since the flat model allows only for non-hardened key derivation, the essential difference to the hierarchical model is that the flat model cannot allow for any secret key leakage as this would render the entire scheme insecure. Hierarchical wallets, on the other hand, introduce hardened nodes whose secret keys can be leaked without affecting the security of any other node in the tree.

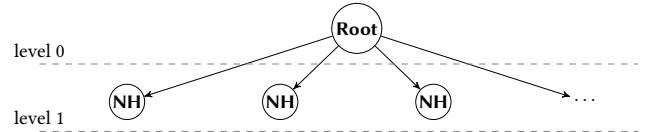


Figure 4: Tree structure of a deterministic wallet scheme in the flat setting.

In the following, we refer to a *tree* as a tuple $(h, n_{0,0}, \mathcal{N}, \mathcal{E})$ if $(\mathcal{N}, \mathcal{E})$ defines a tree of height h with node set $\mathcal{N}$ and edge set $\mathcal{E}$, and a root node $n_{0,0} \in \mathcal{N}$. We denote a directed path p_i^t of length i from the root to a node $n_{i,t} \in \mathcal{N}$ at level i and position t in the tree as the corresponding ordered sequence of edges $p_i^t = (e_1, \dots, e_i) \in \mathcal{E}^i$. A path of length 1 from a node $n_{i-1,s} \in \mathcal{N}$ to a node $n_{i,t} \in \mathcal{N}$ consists of only one edge which we denote as $e_i^{s,t} \in \mathcal{E}$.

Definition 4.1 (Address Structure). Let $\mathcal{T} = (h, n_{0,0}, \mathcal{N}, \mathcal{E})$ be a tree. Define a labeling of the nodes in $\mathcal{N}$ as follows.

- The root node $n_{0,0}$ is labeled by an address $\text{addr}_{0,0}$.
- For $1 \leq t < |\mathcal{N}|$ and $0 \leq i \leq h$, a node $n_{i,t} \in \mathcal{N}$ is labeled by an address $\text{addr}_{i,t} := (\text{addr}_{0,0}, p_i^t)$.

⁴We show in Appendix B that child derivation of hardened nodes is possible under certain conditions.

A tuple $(\mathcal{T}, \text{Addr})$ is said to be an *address structure* (with respect to $\mathcal{T}$) if Addr consists of a set of labels for the nodes in $\mathcal{N}$ that meets the above requirements. A prefix address $\text{addr}_{i,t}^j$ for a node $n_{i,t} \in \mathcal{N}$ with $0 \leq j < i \leq h$ and $t < |\mathcal{N}|$ is a vector of length $j + 1$ consisting of the first $j + 1$ components of $\text{addr}_{i,t} \in \text{Addr}$.

We are now ready to define hierarchical deterministic wallets. In short, these schemes consist of a Setup algorithm, which initializes the root node, hardened and non-hardened secret and public key derivation algorithms $\text{SKDer}_H, \text{PKDer}_H$ and $\text{SKDer}_{NH}, \text{PKDer}_{NH}$ and finally signing and signature verification algorithms Sign and Verify. We assume that public parameters par are known to all parties and we define appropriate secret and public key sets $\mathcal{SK}$ and $\mathcal{PK}$ respectively. We assume there exists a function $\text{ToPubKey} : \mathcal{SK} \rightarrow \mathcal{PK}$ that on input a secret key from $\mathcal{SK}$ outputs the corresponding public key in $\mathcal{PK}$. Formally we have:

Definition 4.2 (Hierarchical Deterministic Wallets). Let $\mathcal{T} = (h, n_{0,0}, \mathcal{N}, \mathcal{E})$ be a tree. A *hierarchical deterministic wallet* scheme is defined w.r.t. an address structure $(\mathcal{T}, \text{Addr})$ and consists of seven algorithms $\text{HDWal} = (\text{Setup}, \text{SKDer}_H, \text{SKDer}_{NH}, \text{PKDer}_H, \text{PKDer}_{NH}, \text{Sign}, \text{Verify})$ which are defined as follows:

- $\text{Setup}(1^\kappa)$: The probabilistic *setup* algorithm takes as input a security parameter 1^κ and outputs a non-hardened master key pair $(\text{msk}_{0,0}, \text{mpk}_{0,0})$ with $\text{msk}_{0,0} \in \mathcal{SK}$, $\text{mpk}_{0,0} \in \mathcal{PK}$ and a chaincode $\text{ch}_{0,0}$.
- $\text{SKDer}_H(\text{sk}_{i,s}, \text{ch}_{i,s}, \text{addr}_{i,s}, e_{i+1}^{s,t})$: The deterministic *hardened secret key derivation* algorithm takes as input a secret key $\text{sk}_{i,s} \in \mathcal{SK}$, a chaincode $\text{ch}_{i,s}$, an address $\text{addr}_{i,s} \in \text{Addr}$ for level $i < h$, positions s, t , as well as an edge $e_{i+1}^{s,t} \in \mathcal{E}$. It outputs a secret key $\text{sk}_{i+1,t} \in \mathcal{SK}$, a chaincode $\text{ch}_{i+1,t}$ and an address $\text{addr}_{i+1,t} \in \text{Addr}$ for level $i + 1$ and position t .
- $\text{SKDer}_{NH}(\text{sk}_{i,s}, \text{pk}_{i,s}, \text{ch}_{i,s}, \text{addr}_{i,s}, e_{i+1}^{s,t})$: The deterministic *non-hardened secret key derivation* algorithm takes as input a secret key $\text{sk}_{i,s} \in \mathcal{SK}$, a public key $\text{pk}_{i,s} \in \mathcal{PK}$, a chaincode $\text{ch}_{i,s}$, an address $\text{addr}_{i,s} \in \text{Addr}$ for level $i < h$, positions s, t , as well as an edge $e_{i+1}^{s,t} \in \mathcal{E}$. It outputs a secret key $\text{sk}_{i+1,t} \in \mathcal{SK}$, a chaincode $\text{ch}_{i+1,t}$ and an address $\text{addr}_{i+1,t} \in \text{Addr}$ for level $i + 1$ and position t .
- $\text{PKDer}_H(\text{sk}_{i,s}, \text{pk}_{i,s}, \text{ch}_{i,s}, \text{addr}_{i,s}, e_{i+1}^{s,t})$: The deterministic *hardened public key derivation* algorithm takes as input a secret key $\text{sk}_{i,s} \in \mathcal{SK}$, a public key $\text{pk}_{i,s} \in \mathcal{PK}$, a chaincode $\text{ch}_{i,s}$, an address $\text{addr}_{i,s} \in \text{Addr}$ for level $i < h$, positions s, t , as well as an edge $e_{i+1}^{s,t} \in \mathcal{E}$. It outputs a public key $\text{pk}_{i+1,t} \in \mathcal{PK}$, a chaincode $\text{ch}_{i+1,t}$ and an address $\text{addr}_{i+1,t} \in \text{Addr}$ for level $i + 1$ and position t .
- $\text{PKDer}_{NH}(\text{pk}_{i,s}, \text{ch}_{i,s}, \text{addr}_{i,s}, e_{i+1}^{s,t})$: The deterministic *non-hardened public key derivation* algorithm takes as input a public key $\text{pk}_{i,s} \in \mathcal{PK}$, a chaincode $\text{ch}_{i,s}$, an address $\text{addr}_{i,s} \in \text{Addr}$ for level $i < h$, positions s, t , as well as an edge $e_{i+1}^{s,t} \in \mathcal{E}$. It outputs a public key $\text{pk}_{i+1,t} \in \mathcal{PK}$, a chaincode $\text{ch}_{i+1,t}$ and an address $\text{addr}_{i+1,t} \in \text{Addr}$ for level $i + 1$ and position t .
- $\text{Sign}(\text{sk}_{i,s}, m)$: The probabilistic *signing* algorithm takes as input a secret key $\text{sk}_{i,s}$ and a message m . It outputs a signature σ .
- $\text{Verify}(\text{pk}_{i,s}, m, \sigma)$: The probabilistic *verification* algorithm takes as input a public key $\text{pk}_{i,s}$, a message m and a signature σ . It outputs 0 or 1.

A hierarchical deterministic wallet is *correct*, if a secret and public key pair is derived correctly using the algorithms $\text{SKDer}_H, \text{PKDer}_H$ or $\text{SKDer}_{NH}, \text{PKDer}_{NH}$, the keys represent a valid signing key pair.

We denote keys with subscript nh (e.g., $\text{sk}_{nh, \cdot}$ or $\text{pk}_{nh, \cdot}$) as *non-hardened* keys and keys with subscript h (e.g., $\text{sk}_{h, \cdot}$ or $\text{pk}_{h, \cdot}$) as *hardened* keys. A key without the subscript nh or h indicates that it can be both a non-hardened or hardened key.

Definition 4.3 (Correctness of Hierarchical Deterministic Wallets). Let HDWal be a hierarchical deterministic wallet scheme with respect to an address structure $(\mathcal{T}, \text{Addr})$. For any $e_1^{0,s} \in \mathcal{E}$ and any $(\text{ch}_{0,0}, \text{msk}_{nh,0,0}, \text{mpk}_{nh,0,0}) \in \text{Setup}(1^\kappa)$, we define tuples $(\text{sk}_{h,1,s}, \text{ch}_{1,s}, \text{addr}_{1,s})$ and $(\text{pk}_{h,1,s}, \text{ch}_{1,s}, \text{addr}_{1,s})$ as

$$\begin{aligned} (\text{sk}_{h,1,s}, \text{ch}_{1,s}, \text{addr}_{1,s}) &:= \text{SKDer}_H(\text{msk}_{nh,0,0}, \text{ch}_{0,0}, \text{addr}_{0,0}, e_1^{0,s}) \\ (\text{pk}_{h,1,s}, \text{ch}_{1,s}, \text{addr}_{1,s}) &:= \text{PKDer}_H(\text{msk}_{nh,0,0}, \text{ch}_{0,0}, \text{addr}_{0,0}, e_1^{0,s}) \end{aligned}$$

and tuples $(\text{sk}_{nh,1,s}, \text{ch}_{1,s}, \text{addr}_{1,s})$ and $(\text{pk}_{nh,1,s}, \text{ch}_{1,s}, \text{addr}_{1,s})$ as

$$\begin{aligned} (\text{sk}_{nh,1,s}, \text{ch}_{1,s}, \text{addr}_{1,s}) &:= \text{SKDer}_{NH}(\text{msk}_{nh,0,0}, \text{ch}_{0,0}, \text{addr}_{0,0}, e_1^{0,s}) \\ (\text{pk}_{nh,1,s}, \text{ch}_{1,s}, \text{addr}_{1,s}) &:= \text{PKDer}_{NH}(\text{mpk}_{nh,0,0}, \text{ch}_{0,0}, \text{addr}_{0,0}, e_1^{0,s}). \end{aligned}$$

Further, for any $\text{addr}_{i-1,s} \in \text{Addr}$, and any edge $e_i^{s,t} \in \mathcal{E}$ we define the tuples $(\text{sk}_{h,i,t}, \text{ch}_{i,t}, \text{addr}_{i,t})$ and $(\text{pk}_{h,i,t}, \text{ch}_{i,t}, \text{addr}_{i,t})$ recursively as

$$\begin{aligned} (\text{sk}_{h,i,t}, \text{ch}_{i,t}, \text{addr}_{i,t}) &:= \text{SKDer}_H(\text{sk}_{nh,i-1,s}, \text{ch}_{i-1,s}, \text{addr}_{i-1,s}, e_i^{s,t}) \\ (\text{pk}_{h,i,t}, \text{ch}_{i,t}, \text{addr}_{i,t}) &:= \text{PKDer}_H(\text{sk}_{nh,i-1,s}, \text{ch}_{i-1,s}, \text{addr}_{i-1,s}, e_i^{s,t}) \end{aligned}$$

and tuples $(\text{sk}_{nh,i,t}, \text{ch}_{i,t}, \text{addr}_{i,t})$ and $(\text{pk}_{nh,i,t}, \text{ch}_{i,t}, \text{addr}_{i,t})$ as

$$\begin{aligned} (\text{sk}_{nh,i,t}, \text{ch}_{i,t}, \text{addr}_{i,t}) &:= \text{SKDer}_{NH}(\text{sk}_{nh,i-1,s}, \text{ch}_{i-1,s}, \text{addr}_{i-1,s}, e_i^{s,t}) \\ (\text{pk}_{nh,i,t}, \text{ch}_{i,t}, \text{addr}_{i,t}) &:= \text{PKDer}_{NH}(\text{pk}_{nh,i-1,s}, \text{ch}_{i-1,s}, \text{addr}_{i-1,s}, e_i^{s,t}) \end{aligned}$$

HDWal is *correct* if for all $m \in \{0, 1\}^*$, all $1 \leq i \leq h$, all $1 \leq t \leq (1-d^{h+1})/(1-d)$, and all $(\text{ch}_{0,0}, \text{msk}_{nh,0,0}, \text{mpk}_{nh,0,0}) \in \text{Setup}(1^\kappa)$ it holds that

$$\begin{aligned} &\Pr_{\sigma \leftarrow \text{Sign}(\text{sk}_{h,i,t}, m)} [\text{Verify}(\text{pk}_{h,i,t}, \sigma, m) = 1] = 1 \\ &\wedge \Pr_{\sigma \leftarrow \text{Sign}(\text{sk}_{nh,i,t}, m)} [\text{Verify}(\text{pk}_{nh,i,t}, \sigma, m) = 1] = 1. \end{aligned}$$

4.1 Oracles

Let us now describe the general capability and influence that the adversary has over the hierarchical wallet schemes. An adversary is allowed to create new hardened and non-hardened nodes in the tree. Furthermore, the adversary can corrupt the hot wallet of all non-hardened nodes, thereby learning the public key and the chaincode of these nodes, as well as learning the secret key and chaincode of the hardened nodes. As we mentioned earlier, since hardened keys are given to untrustworthy nodes, the adversary is able to corrupt both their hot and cold wallets and as such, we do not consider the hardened nodes to derive new children. One way to look at hardened nodes, is that such nodes are the root of a new tree. We will later show in App. B that an adversary cannot distinguish hardened key pairs from freshly generated keys except with negligible probability. Therefore, our model can be

recursively extended to consider settings where the hardened nodes can also derive new keys. Finally, the adversary can query any node on a freely chosen message m and receive a signature for this message. To model the above mentioned capabilities, we describe the oracles which the adversary gets access to in the unlinkability game $\text{unl}_{\text{HDWal}}$ and the unforgeability game $\text{wufcma1}_{\text{HDWal}}$.

Initially, two lists $\text{SK} = \emptyset$ and $\text{CH} = \emptyset$ are initialized. These are used throughout the oracles to bookkeep which secret keys and chaincodes have been leaked to the adversary. In the following, we consider a fixed address structure $(\mathcal{T}, \text{Addr})$.

- **Hardened Child Creation** HChild0 : On inputs an address $\text{addr}_{i,s}$ and an edge $e_{i+1}^{s,t}$ from $\mathcal{A}$, return $\perp$ if the address $\text{addr}_{i,s}$ belongs to a hardened node or the address $\text{addr}_{i,s}$ is not valid (i.e., $\text{addr}_{i,s} \notin \text{Addr}$). Further, return $\perp$, if the address $\text{addr}_{i+1,t}$ exists already. Otherwise, compute the keys and chaincode $(\text{sk}_{h,i,s}, \text{pk}_{h,i,s})$ and $\text{ch}_{i,s}$ for the node $\text{addr}_{i,s}$ by recursively deriving keys along the path in the tree, starting from the first node in the path that has already been assigned a key. Create a hardened child with address $\text{addr}_{i+1,t}$ as follows. Generate keypair $(\text{sk}_{h,i+1,t}, \text{pk}_{h,i+1,t})$ by executing both secret and public key derivation algorithms.

$$(\text{sk}_{h,i+1,t}, \text{ch}_{i+1,t}, \text{addr}_{i+1,t}) \leftarrow \text{SKDer}_H(\text{sk}_{h,i,s}, \text{ch}_{i,s}, \text{addr}_{i,s}, e_{i+1}^{s,t})$$

$$(\text{pk}_{h,i+1,t}, \text{ch}_{i+1,t}, \text{addr}_{i+1,t}) \leftarrow \text{PKDer}_H(\text{sk}_{h,i,s}, \text{ch}_{i,s}, \text{addr}_{i,s}, e_{i+1}^{s,t}).$$

Return $\text{pk}_{h,i+1,t}$.

- **Non-Hardened Child Creation** NHChild0 : On inputs an address $\text{addr}_{i,s}$ and an edge $e_{i+1}^{s,t}$ from $\mathcal{A}$, return $\perp$ if the address $\text{addr}_{i,s}$ belongs to a hardened node or the address $\text{addr}_{i,s}$ is not valid (i.e., $\text{addr}_{i,s} \notin \text{Addr}$). Further, return $\perp$, if the address $\text{addr}_{i+1,t}$ exists already. Otherwise, compute the keys and chaincode $(\text{sk}_{h,i,s}, \text{pk}_{h,i,s})$ and $\text{ch}_{i,s}$ for the node $\text{addr}_{i,s}$ by recursively deriving keys along the path in the tree, starting from the first node in the path that has already been assigned a key. Create a non-hardened child with address $\text{addr}_{i+1,t}$ as follows. Generate keypair $(\text{sk}_{nh,i+1,t}, \text{pk}_{nh,i+1,t})$ by executing both key derivation algorithms

$$(\text{sk}_{nh,i+1,t}, \text{ch}_{i+1,t}, \text{addr}_{i+1,t}) \leftarrow \text{SKDer}_{\text{NH}}(\text{sk}_{h,i,s}, \text{ch}_{i,s}, \text{addr}_{i,s}, e_{i+1}^{s,t})$$

$$(\text{pk}_{nh,i+1,t}, \text{ch}_{i+1,t}, \text{addr}_{i+1,t}) \leftarrow \text{PKDer}_{\text{NH}}(\text{pk}_{h,i,s}, \text{ch}_{i,s}, \text{addr}_{i,s}, e_{i+1}^{s,t}).$$

Return $\text{pk}_{nh,i+1,t}$.

- **Signing** HDSign0 : On input message m and an address $\text{addr}_{i,s}$ from $\mathcal{A}$, proceed as follows. Return $\perp$ if the address $\text{addr}_{i,s}$ is not valid (i.e., $\text{addr}_{i,s} \notin \text{Addr}$). Further, check if $\text{addr}_{i,s}$ has already been queried to either NHChild0 or HChild0 and return $\perp$ if this is not the case. Let $\text{sk}_{i,s}$ be the secret key for the node with address $\text{addr}_{i,s}$. Then compute a signature $\sigma \leftarrow \text{Sign}(\text{sk}_{i,s}, m)$, add m to the message list $\text{SigList}[\text{addr}_{i,s}]$ and return σ .⁵
- **Chaincode Leakage** CHLeak0 : On input an address $\text{addr}_{i,s}$ from $\mathcal{A}$, check if $\text{addr}_{i,s}$ has already been queried to either NHChild0 or HChild0 and return $\perp$ if this is not the case. Set $\text{CH}[\text{addr}_{i,s}] = 1$ to denote that the chaincode $\text{ch}_{i,s}$ of address $\text{addr}_{i,s}$ has been leaked and return $(\text{pk}_{i,s}, \text{ch}_{i,s})$.
- **Secret Key Leakage (for hardened node)** SKLeak0 : On input an address $\text{addr}_{i,s}$ from $\mathcal{A}$, check if the address is that of the root, i.e., $\text{addr}_{i,s} = \text{addr}_{0,0}$ or if the address belongs to a non-hardened

node; in this case, return $\perp$. Further, check if $\text{addr}_{i,s}$ has already been queried to either NHChild0 or HChild0 and return $\perp$ if this is not the case. Else, set $\text{SK}[\text{addr}_{i,s}] = 1$ and $\text{CH}[\text{addr}_{i,s}] = 1$ to denote that the secret key $\text{sk}_{h,i,s}$ and the chaincode $\text{ch}_{i,s}$ of address $\text{addr}_{i,s}$ have been leaked and return $(\text{sk}_{h,i,s}, \text{ch}_{i,s})$.

4.2 Unlinkability

Intuitively, the notion of unlinkability for hierarchical deterministic wallets guarantees that public keys in the tree, i.e., public keys that have been derived directly or indirectly from the master key of the tree root, cannot be distinguished from from a freshly generated public key. More concretely, the distribution of public keys from the tree should be computationally indistinguishable from a distribution of public keys that have been derived from an independently chosen master key. While this is a valuable privacy notion, it does not quite model practical scenarios in the hot/cold wallet setting. Recall that this setting assumes public keys and chaincodes to be stored in hot wallets, which are prone to corruptions. Therefore, we extend the unlinkability notion as described above in the following way. We consider hot wallet corruption upon which the public key and chaincode of the corrupted wallet are leaked. This extended notion gives more power to the adversary and is more close to the capabilities that an adversary has in real life scenarios. Naturally, the adversary can distinguish the distribution of keys derived from public keys of corrupted hot wallets from a distribution of public keys that have been derived from an independently chosen master key. Therefore, in our new unlinkability notion the adversary should not be able to distinguish the distribution of keys derived from non-compromised hot wallets and keys derived from independently chosen master keys.

In the following we describe the unlinkability game $\text{unl}_{\text{HDWal}}$ with respect to a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$. In the first step of the game, the challenger generates a fresh master key pair and a chaincode via the execution of $\text{Setup}(1^\kappa)$. The adversary receives the master public key as input and obtains access to all oracles as described in subsection 4.1. At some point, the adversary outputs an address $\text{addr}_{i,s}$ and an edge $e_{i+1}^{s,t}$ and receives a public key from the challenger. This public key is either the correct key for the node at address $\text{addr}_{i,s}$ or a public key derived for a random address from a fresh master public key. $\mathcal{A}$ wins the game if it can successfully distinguish these two scenarios. In the following we give a detailed description of the game $\text{unl}_{\text{HDWal}}$:

Game $\text{unl}_{\text{HDWal}}$:

- **Setup Phase:** The challenger computes $(\text{ch}_{0,0}, \text{msk}_{0,0}, \text{mpk}_{0,0}) \leftarrow \text{Setup}(1^\kappa)$ and sends $\text{mpk}_{0,0}$ to $\mathcal{A}$.
- **Online Phase:** On input the security parameter and the master public key $\text{mpk}_{0,0}$, the adversary $\mathcal{A}$ is allowed to make queries to the oracles as explained in subsection 4.1.
- **Output Phase:** Eventually, $\mathcal{A}$ chooses an address $\text{addr}_{i,s}$, an edge $e_{i+1}^{s,t}$ and a value $c \in \{h, nh\}$ and sends them to the challenger. Let $(\text{sk}_{i,s}, \text{pk}_{i,s})$ be the key pair and $\text{ch}_{i,s}$ the chaincode of the node at address $\text{addr}_{i,s}$. If the address $\text{addr}_{i,s}$ belongs to a hardened node, $\mathcal{C}$ returns $\perp$. Otherwise, the challenger chooses a bit $b \xleftarrow{\$} \{0, 1\}$ and generates a public key $\text{pk}_{i+1,t}$ as follows:
 - If $b = 0$:
 - * If $c = h$: $\mathcal{C}$ computes $(\text{pk}_{h,i+1,t}, \cdot, \cdot) \leftarrow \text{PKDer}_H(\text{sk}_{h,i,s}, \text{ch}_{i,s}, \text{addr}_{i,s}, e_{i+1}^{s,t})$.

⁵In case of one-per message unforgeability, the oracle aborts if it has been queried previously on input $(m, \text{addr}_{i,s})$.

- * **If $c = nh$:** If the chaincode for $\mathbf{addr}_{i,s}$ or any of its prefix addresses has been leaked, i.e., $\text{CH}[\mathbf{addr}_{i,s}^j] = 1$, for any $j < i$, then C returns $\perp$. Else, C computes $(\text{pk}_{nh,i+1,t}, \cdot, \cdot) \leftarrow \text{PKDer}_{\text{NH}}(\text{pk}_{nh,i,s}, \text{ch}_{i,s}, \mathbf{addr}_{i,s}, \mathbf{e}_{i+1}^{s,t})$.
- If $b = 1$: The challenger computes $(\text{ch}'_{0,0}, \text{msk}'_{0,0}, \text{mpk}'_{0,0}) \leftarrow \text{Setup}(1^\kappa)$.
- * **If $c = h$:** C derives a public key $\text{pk}'_{h,1,t} \leftarrow \text{PKDer}_H(\text{msk}'_{0,0}, \text{ch}'_{0,0}, \mathbf{addr}_{0,0}, \mathbf{e}_1^{0,t})$.
- * **If $c = nh$:** If the chaincode for $\mathbf{addr}_{i,s}$ or any of its prefix addresses has been leaked, i.e., $\text{CH}[\mathbf{addr}_{i,s}^j] = 1$, for any $j < i$, then C returns $\perp$. Else C derives a public key $\text{pk}'_{nh,1,t} \leftarrow \text{PKDer}_{\text{NH}}(\text{mpk}'_{0,0}, \text{ch}'_{0,0}, \mathbf{addr}_{0,0}, \mathbf{e}_1^{0,t})$.
- Based on the value of b and c , the challenger sends to the adversary either $\text{pk}_{h,i+1,t}$ or $\text{pk}_{nh,i+1,t}$ or $\text{pk}'_{h,1,t}$ or $\text{pk}'_{nh,1,t}$.
- The adversary can continue to make oracle queries under the restrictions as mentioned above.
- Eventually, $\mathcal{A}$ outputs a bit b' and wins the game if $b = b'$.

We define the advantage of an adversary $\mathcal{A}$ in $\text{unl}_{\text{HDWal}}$ as

$$\text{Adv}_{\text{unl}_{\text{HDWal}}}^{\mathcal{A}} := \left| \Pr[\text{unl}_{\text{HDWal}}^{\mathcal{A}} = 1] - \frac{1}{2} \right|.$$

On Forward Unlinkability. The model of hierarchical wallets as defined in Definition 4.2 in Section 4 is stateless. In other words, each node in the tree maintains a fixed chaincode $\text{ch}_{i,s}$ which is used as an input parameter for the child key derivation algorithms. If the (non-hardened) public key $\text{pk}_{i,s}$ as well as the chaincode $\text{ch}_{i,s}$ of a node are leaked (e.g., due to a hot wallet corruption of the node in the hot/cold wallet setting), then the adversary can as well compute the non-hardened keys in the entire sub-tree under $\text{pk}_{i,s}$. Consequently, unlinkability of the sub-tree is lost. To enhance the unlinkability property, we can extend our model to a stateful variant where, each node maintains a state $\text{St}_{i,s}^t$. On every child key derivation, the state of the node is refreshed to a new state $\text{St}_{i,s}^{t+1}$. As a result of this modification, we can guarantee *forward unlinkability* for hierarchical wallets, which is similar to the standard notion of *forward security*. Precisely, on a hot wallet corruption, the adversary learns the *current state* $\text{St}_{i,s}^t$ and the public key $\text{pk}_{i,s}$ of a node. However, the existing children of this node were derived from earlier states $\text{St}_{i,s}^{t'}$, for $t' < t$ - which are not known to the adversary. Thus it can no longer break the unlinkability of the existing child keys in the sub-tree under $\text{pk}_{i,s}$. However, it would be able to link any future child keys derived from $\text{pk}_{i,s}$.

4.3 Unforgeability

The notion of unforgeability for hierarchical deterministic wallets in the hot/cold wallet setting guarantees that an adversary cannot forge a signature of any uncorrupted node in the tree. In our model, non-hardened keys are always stored in hot/cold wallets, i.e., the secret keys are secured in the cold wallet storage, which cannot be corrupted by an adversary. Hardened keys, on the other hand, can be stored on any device and are thereby prone to corruption. Therefore, we allow an adversary to corrupt hardened secret keys, while non-hardened secret keys must remain uncorrupted.

In more detail, the unforgeability game proceeds as follows. The challenger generates a master key pair and a chaincode via the execution of $\text{Setup}(1^\kappa)$. The adversary receives the master public key and obtains access to the oracles as described in subsection 4.1. Eventually, the adversary outputs a forgery, i.e., a message and a signature for a specific node in the tree. The adversary wins the game, if the signature is valid, the message has not been queried to the signing oracle HDSigN0 for this specific node before and the cold wallet of the node is uncorrupted. We note that a slightly weaker variant of unforgeability for hierarchical deterministic wallets is the notion of *one-per message unforgeability*, where the security game proceeds exactly as the game of the unforgeability notion with the difference that the adversary is allowed to query the HDSigN0 oracle only once for each message/address pair. We now give a detailed description of the unforgeability game $\text{wufcma1}_{\text{HDWal}}$.

Game $\text{wufcma1}_{\text{HDWal}}$:

- **Setup Phase:** The challenger computes $(\text{ch}_{0,0}, \text{msk}_{0,0}, \text{mpk}_{0,0}) \leftarrow \text{Setup}(1^n)$ and sends $\text{ch}_{0,0}$ and $\text{mpk}_{0,0}$ to $\mathcal{A}$.
- **Online Phase:** On input the security parameter, the adversary $\mathcal{A}$ is allowed to make queries to the oracles as explained in subsection 4.1.
- **Output Phase:** Eventually, $\mathcal{A}$ outputs a public key pk_{i^*,s^*} , a message m^* , an address $\mathbf{addr}_{i^*,s^*}$ and a signature σ^* . $\mathcal{A}$ wins if all of the following conditions hold,
 - $\text{Verify}(\text{pk}_{i^*,s^*}, \sigma^*, m^*) = 1$
 - $m^* \notin \text{SigList}[\mathbf{addr}_{i^*,s^*}]$
 - Either $\mathbf{addr}_{i^*,s^*}$ belongs to a non-hardened node or $\mathbf{addr}_{i^*,s^*}$ belongs to a hardened node and its secret key has not been corrupted, i.e., $\text{SK}[\mathbf{addr}_{i^*,s^*}] = 0$.

We define the advantage of an adversary $\mathcal{A}$ in $\text{wufcma1}_{\text{HDWal}}$ as

$$\text{Adv}_{\text{unl}_{\text{HDWal}}}^{\mathcal{A}} := \Pr[\text{wufcma1}_{\text{HDWal}}^{\mathcal{A}} = 1].$$

5 GENERIC CONSTRUCTION

In this section, we first show how to generically construct a hierarchical deterministic wallet scheme HDWal from a signature scheme with perfectly rerandomizable keys $\text{RSig} = (\text{RSig.Gen}, \text{RSig.RandSK}, \text{RSig.RandPK}, \text{RSig.Sign}, \text{RSig.Verify})$. We denote the construction of HDWal with respect to a signature scheme with rerandomizable keys RSig by $\text{HDWal}[\text{RSig}]$. Our generic construction $\text{HDWal}[\text{RSig}]$ uses internally a hash function $H : \{0, 1\}^* \rightarrow \mathcal{R} \times \{0, 1\}^\kappa$. We detail our construction in Figure 5. Subsequently, we analyze the security of our generic construction by proving the unlinkability and the unforgeability properties of $\text{HDWal}[\text{RSig}]$. Due to space limitations, we present the full proofs for unlinkability and unforgeability of $\text{HDWal}[\text{RSig}]$ in Appendices B and C. In the following subsection, we present the theorem that states that $\text{HDWal}[\text{RSig}]$ satisfies $\text{wufcma1}_{\text{HDWal}}$ security with a loss in the security reduction. We then show that this loss is indeed unavoidable which means that our security reduction is optimal.

5.1 Unforgeability of Generic Construction

We now analyze the unforgeability property of our generic construction $\text{HDWal}[\text{RSig}]$ of a hierarchical wallet. We require the following properties from the underlying signature scheme RSig .

```

Algorithm HDWal[RSig].Setup(par)
00  $ch_{0,0} \xleftarrow{\$} \{0,1\}^K$ 
01  $(msk_{0,0}, mpk_{0,0}) \xleftarrow{\$} \text{RSig.Gen}(\text{par})$ 
02 Return  $(msk_{0,0}, mpk_{0,0}, ch_{0,0})$ 

Algorithm HDWal[RSig].Sign( $sk_{i,s}, m$ )
00  $\sigma \leftarrow \text{RSig.Sign}(sk_{i,s}, m)$ 
01 Return  $\sigma$ 

Algorithm HDWal[RSig].Verify( $pk_{i,s}, \sigma, m$ )
00  $0/1 \leftarrow \text{RSig.Verify}(pk_{i,s}, \sigma, m)$ 
01 Return  $0/1$ 

Algorithm HDWal[RSig].SKDerH( $sk_{i,s}, ch_{i,s}, \text{addr}_{i,s}, e_{i+1}^{s,t}$ )
00  $(\omega, ch_{i+1,t}) \leftarrow H(sk_{i,s}, ch_{i,s}, e_{i+1}^{s,t})$ 
01  $sk_{i+1,t} \leftarrow \text{RSig.RandSK}(sk_{i,s}; \omega)$ 
02  $\text{addr}_{i+1,t} \leftarrow \text{addr}_{i,s} \parallel e_{i+1}^{s,t}$ 
03 Return  $(sk_{i+1,t}, ch_{i+1,t}, \text{addr}_{i+1,t})$ 

Algorithm HDWal[RSig].SKDerNH( $sk_{i,s}, pk_{i,s}, ch_{i,s}, \text{addr}_{i,s}, e_{i+1}^{s,t}$ )
00  $(\omega, ch_{i+1,t}) \leftarrow H(pk_{i,s}, ch_{i,s}, e_{i+1}^{s,t})$ 
01  $sk_{i+1,t} \leftarrow \text{RSig.RandSK}(sk_{i,s}; \omega)$ 
02  $\text{addr}_{i+1,t} \leftarrow \text{addr}_{i,s} \parallel e_{i+1}^{s,t}$ 
03 Return  $(sk_{i+1,t}, ch_{i+1,t}, \text{addr}_{i+1,t})$ 

Algorithm HDWal[RSig].PKDerH( $sk_{i,s}, pk_{i,s}, ch_{i,s}, \text{addr}_{i,s}, e_{i+1}^{s,t}$ )
00  $(\omega, ch_{i+1,t}) \leftarrow H(sk_{i,s}, ch_{i,s}, e_{i+1}^{s,t})$ 
01  $pk_{i+1,t} \leftarrow \text{RSig.RandPK}(pk_{i,s}; \omega)$ 
02  $\text{addr}_{i+1,t} \leftarrow \text{addr}_{i,s} \parallel e_{i+1}^{s,t}$ 
03 Return  $(pk_{i+1,t}, ch_{i+1,t}, \text{addr}_{i+1,t})$ 

Algorithm HDWal[RSig].PKDerNH( $pk_{i,s}, ch_{i,s}, \text{addr}_{i,s}, e_{i+1}^{s,t}$ )
00  $(\omega, ch_{i+1,t}) \leftarrow H(pk_{i,s}, ch_{i,s}, e_{i+1}^{s,t})$ 
01  $pk_{i+1,t} \leftarrow \text{RSig.RandPK}(pk_{i,s}; \omega)$ 
02  $\text{addr}_{i+1,t} \leftarrow \text{addr}_{i,s} \parallel e_{i+1}^{s,t}$ 
03 Return  $(pk_{i+1,t}, ch_{i+1,t}, \text{addr}_{i+1,t})$ 

```

Figure 5: Generic construction of a hierarchical deterministic wallet scheme HDWal[RSig] from a signature with perfectly rerandomizable keys RSig. HDWal[RSig] is defined w.r.t. an address structure $(\mathcal{T}, \text{Addr})$, where $\mathcal{T} = (h, n_{0,0}, \mathcal{N}, \mathcal{E})$, such that $\text{addr}_{i,s} \in \text{Addr}$ and $e_i^{s,t} \in \mathcal{E}$ for $0 \leq i \leq h$ and $1 \leq s, t \leq |\mathcal{N}|$. We denote by $(pk_{i,s}, sk_{i,s})$ and $ch_{i,s}$ the public/secret key pair and chaincode of the node with address $\text{addr}_{i,s}$. We denote by H a hash function $H: \{0,1\}^* \rightarrow \mathcal{R} \times \{0,1\}^K$.

RSig must satisfy (1) the definition of a signature scheme with rerandomizable keys as well as (2) a transitive property of the keys. We formally define the latter below.

Definition 5.1 (Transitive Rerandomization). Let $\text{RSig} = (\text{RSig.Gen}, \text{RSig.Sign}, \text{RSig.Verify}, \text{RSig.RandSK}, \text{RSig.RandPK})$ be a signature scheme with perfectly rerandomizable keys. We say that RSig *transitively rerandomizes* if there exists an operation $\odot: \mathcal{R} \times \mathcal{R} \rightarrow \mathcal{R}$ s.t. for all $(sk, pk) \in \text{RSig.Gen}(\text{par})$ and all $(\rho, \rho') \in \mathcal{R} \times \mathcal{R}$, the values (sk', pk') , (sk'', pk'') , $\tilde{\rho}$ which are defined as

$$\begin{aligned}
 (sk', pk') &\leftarrow (\text{RSig.RandSK}(sk; \rho), \text{RSig.RandPK}(pk; \rho)) \\
 (sk'', pk'') &\leftarrow (\text{RSig.RandSK}(sk'; \rho'), \text{RSig.RandPK}(pk'; \rho')), \\
 \tilde{\rho} &= \rho \odot \rho' \text{ satisfy}
 \end{aligned}$$

$$(sk'', pk'') = (\text{RSig.RandSK}(sk; \tilde{\rho}), \text{RSig.RandPK}(pk; \tilde{\rho})).$$

Definition 5.2 (Invertible Rerandomization). Let $\text{RSig} = (\text{RSig.Gen}, \text{RSig.Sign}, \text{RSig.Verify}, \text{RSig.RandSK}, \text{RSig.RandPK})$ be a signature scheme with perfectly rerandomizable keys. We say that RSig has *invertible rerandomization* if there exist (efficient) algorithms RandSK^{-1} and RandPK^{-1} s.t. for all $(sk, pk) \in \text{RSig.Gen}(\text{par})$ and all $\rho \in \mathcal{R}$ it holds

$$\begin{aligned}
 sk &= \text{RandSK}^{-1}(\text{RSig.RandSK}(sk; \rho); \rho) \\
 pk &= \text{RandPK}^{-1}(\text{RSig.RandPK}(pk; \rho); \rho)
 \end{aligned}$$

We note that the signature schemes with rerandomizable keys based on Schnorr [20], BLS [15] and ECDSA (additive variant presented in Section 3 of this work and multiplicative variant presented in [15]) all satisfy the properties of transitive rerandomization and invertible rerandomization as defined in Definitions 5.1, 5.2. For the Schnorr, BLS and additive ECDSA based schemes, the $\odot$ operation is a simple addition, while for the multiplicative ECDSA scheme it is a multiplication (modulo the group order p). Below we state our theorem for the one-per message unforgeability property of HDWal[RSig].

Theorem 5.3 Let HDWal[RSig] be the construction defined in Figure 5, let $H: \{0,1\}^* \rightarrow \mathcal{R} \times \{0,1\}^K$ be a hash function modeled as a random oracle and let RSig be a signature scheme with rerandomizable keys that satisfies the property of transitive rerandomization and invertible rerandomization as in Definitions 5.1, 5.2. Let $\mathcal{A}$ be an adversary playing in the game $\text{wufcma1}_{\text{HDWal[RSig]}}^{\mathcal{A}}$, then there exists an algorithm C running in roughly the same time as $\mathcal{A}$, and that makes as many queries to the oracle Rand in uf-cma-hrk1 as $\mathcal{A}$ makes queries to NHChild0/HChild0 such that

$$\text{Adv}_{\text{uf-cma-hrk1}_{\text{RSig}}}^C \geq \frac{1}{4e(q_{sk} + 1)} \cdot \text{Adv}_{\text{wufcma1}_{\text{HDWal[RSig]}}^{\mathcal{A}}}.$$

where q_{sk} is the number of SKLeak0 oracle queries from $\mathcal{A}$.

We stated Theorem 5.3 w.r.t. the one-per message unforgeability notions of hierarchical deterministic wallet schemes and signature schemes with rerandomizable keys, because these notions are sufficient in the setting of deterministic wallets. This is because wallets sign each unique transaction at most once. However, we note that we can likewise state and prove the above theorem with respect to the standard unforgeability notions, i.e., the notions that do not restrict the adversary to obtain at most one signature on a specific message.

We provide the full proof of Theorem 5.3 in Appendix C. Essentially, the proof exhibits an adversary C who uses the adversary $\mathcal{A}$ who plays in game $\text{wufcma1}_{\text{HDWal[RSig]}}^{\mathcal{A}}$ to win its own game $\text{uf-cma-hrk1}_{\text{RSig}}^C$. The main idea of our proof is that C guesses in advance which hardened nodes $\mathcal{A}$ might corrupt (i.e., calls the SKLeak0 oracle on). In case the guess of C is wrong, C cannot answer all SKLeak0 oracle queries from $\mathcal{A}$ and therefore has to abort. This leads to a polynomial loss in the number of SKLeak0 oracle queries (i.e., q_{sk}) in C 's advantage in its $\text{uf-cma-hrk1}_{\text{RSig}}^C$ game. We use Coron's technique as presented in [13] to bound this loss.

Interestingly, the following theorem states that this loss in the advantage of C is inherent and that, in fact, there does not exist a

tighter security reduction. In Appendix D, we recall the security notion of *unforgeability under rerandomized keys* $\text{uf-cma-rk}_{\text{RSig}}$ for a signature scheme with rerandomizable keys RSig as introduced in [20] and prove Theorem 5.4. Below, we denote as $\mathcal{A}_1^{\mathcal{A}_2}$ that $\mathcal{A}_1$ has black-box access to $\mathcal{A}_2$. In particular, it does not rewind $\mathcal{A}_2$.

Theorem 5.4 *Let HDWal be an algorithm such that for any signature scheme with rerandomizable keys RSig , $\text{HDWal}^{\text{RSig}}$ is a hierarchical deterministic wallet scheme. Moreover, suppose that there is a reduction $\mathcal{R}$ such that for every signature scheme with rerandomizable keys RSig and every adversary $\mathcal{A}$ running in time $t_{\mathcal{A}}$ with $\epsilon_{\mathcal{A}} = \text{Adv}_{\text{wufcma1}_{\text{HDWal}^{\text{RSig}}}}^{\mathcal{A}}$, it holds that $\text{Adv}_{\text{uf-cma-rk}_{\text{RSig}}}^{\mathcal{R}^{\mathcal{A}}} \geq \epsilon_{\mathcal{R}}$ and $\mathcal{R}^{\mathcal{A}}$ runs in time $t_{\mathcal{R}}$. Then there exists an algorithm $\mathcal{M}$ running in time $t_{\mathcal{M}} \leq 2 \cdot t_{\mathcal{R}}$ s.t. $\text{Adv}_{\text{uf-cma-rk}_{\text{RSig}}}^{\mathcal{M}} \geq \epsilon_{\mathcal{R}} - \epsilon_{\mathcal{A}} \cdot \frac{2 \exp(-1)}{q_{\text{sk}}}$.*

Theorem 5.4 implies that if there exists a reduction from $\text{uf-cma-rk}_{\text{RSig}}$ to $\text{wufcma1}_{\text{HDWal}^{\text{RSig}}}$ for a signature scheme with rerandomizable keys RSig s.t. the reduction loses less than a factor proportional to q_{sk} , then there exists an efficient algorithm $\mathcal{M}$ that can break the $\text{uf-cma-rk}_{\text{RSig}}$ security. We formulate and prove this result w.r.t. a reduction from the strongest possible security notion of signature schemes with rerandomizable keys (i.e., uf-cma-rk) to the restricted notion of one-per message wallet unforgeability (i.e., $\text{wufcma1}_{\text{HDWal}}$). Clearly, this implies that the result from Theorem 5.4 also holds for the weaker notion of uf-cma-hrk1 for signature schemes with rerandomizable keys which we use in Theorem 5.3. We note that Theorem 5.4 can likewise be stated and proved with respect to the standard unforgeability notion of hierarchical deterministic wallet schemes, i.e., the notion that does not restrict the adversary to obtain at most one signature on a specific message.

6 DISCUSSION

On Security Parameters We instantiate our generic hierarchical deterministic wallet construction $\text{HDWal}[\text{RSig}]$ with two schemes, namely $\text{REC}[\text{H}_1]$ (Figure 7) and $\text{REC}'[\text{H}_1]$ (Figure 6). Note that $\text{HDWal}[\text{REC}[\text{H}_1]]$ corresponds to the BIP32 wallet, while $\text{HDWal}[\text{REC}'[\text{H}_1]]$ is instantiated from the multiplicatively rerandomized construction $\text{REC}'[\text{H}_1]$ from [15], we will refer to it as BIP32-m.

First, let us recall, how to compute the bit security level of a scheme. A hierarchical wallet scheme HDWal is said to have a bit security level of κ bits, if any algorithm $\mathcal{A}$ with running time t and advantage ϵ in $\text{wufcma1}_{\text{HDWal}}$ takes *expected* running time $\frac{t}{\epsilon} \geq 2^{\kappa}$ to break the scheme for the first time. (The security level for a conventional signature scheme is defined analogously). From our Theorems E.1, E.2, we compute the bit security level of our schemes, considering an algorithm $\mathcal{A}$ with parameters t', ϵ' (in game $\text{wufcma1}_{\text{HDWal}}$), where $t' \approx t$ and $\epsilon' = \epsilon \cdot Q$ for some $Q \geq 1$ and where t, ϵ denote the runtime and advantage of the related forger $\mathcal{C}$ in game $\text{uf-cma1}_{\text{EC}}$. By assumption, EC satisfies $\kappa = 128$ bits of security, hence $\frac{t}{\epsilon} \geq 2^{128}$. Thus, we obtain $\frac{t'}{\epsilon'} = \frac{t}{\epsilon \cdot Q} \geq \frac{2^{128}}{Q} = 2^{\kappa - \log Q}$. Our results are reported in Table 1, where we took an estimate of the practical parameters as follows: the total number of keys is $q = 2^{20}$, the number of q_{sk} of secret keys leaked is roughly 1% of the total number of keys q , i.e., $q_{\text{sk}} \approx 2^{14}$.

Scheme	Theorem Ref.	Bit Security with $\kappa = 128$
BIP32	Thm E.1	$\log(Q) = \log(q \cdot 4e \cdot q_{\text{sk}}) \approx 37, \kappa - \log(Q) = 91$
BIP32-m	Thm E.2	$\log(Q) = \log(4e \cdot q_{\text{sk}}) \approx 17, \kappa - \log(Q) = 111$

Table 1: Bit Security Level of BIP32 and BIP32-m, relying on uf-cma1 of EC[H₀]

On BIP32 Parameters. Our construction of $\text{HDWal}[\text{REC}[\text{H}_1]]$ gives us the BIP32 construction as specified in [33]. Here we list the exact parameters used in BIP32 and minor differences of BIP32 with our construction $\text{HDWal}[\text{REC}[\text{H}_1]]$.

- Each node can derive at most 2^{32} children nodes.
- e_{pk} is chosen from $\{0, 1\}^{32}$, which allows each non-hardened node to generate 2^{31} non-hardened and 2^{31} hardened child keys.
- A child key is derived as a hardened or a non-hardened node based on whether $e_{\text{pk}} \geq 2^{31}$ or $\leq 2^{31}$ respectively. However, this is syntactical, and does not affect our security analysis.
- Although at each level, the total number of derived keys can be at most $(2^{32}) \cdot p$, where p is the number of parent nodes in the immediate upper level, we do not imagine that all of these keys are derived at every level. As can be seen, this would already exceed our parameter $q = 2^{20}$, as selected above.
- The chaincode ch_{pk} is chosen from $\{0, 1\}^{256}$.
- The input parameter addr_{pk} to the key derivation algorithms is set to an empty string. We use this parameter to indicate the position in the tree, at which the child key is derived and to ensure that the actual BIP32 derivation algorithms are called on the proper inputs for this position.
- The input parameter addr_{pk} to the key derivation algorithms is set to an empty string λ . Let us briefly explain this syntactical difference. In our Definition 4.2, $\text{addr}_{\text{pk}} \neq \lambda$ is provided as input. This makes the user aware of the position in the tree, at which the child key is derived and makes sure that the actual BIP32 derivation algorithms are called on the proper inputs for this position in the tree.

Open Questions Finally, let us mention some interesting open questions that can be answered in future works:

- Is it possible to remove the one per-message restriction and prove the security of the additively rerandomizable ECDSA scheme in the uf-cma-hrk notion? Additionally, is there a tight reduction to uf-cma-hrk ?
- Can we improve the tightness of uf-cma1 security [18] of ECDSA from the semi-logarithm problem?

ACKNOWLEDGMENTS

The authors are grateful to the anonymous reviewers for their valuable comments. This work is supported by the German Research Foundation (DFG) Emmy Noether Program FA 1320/1-1, by the German Research Foundation DFG - SFB 1119 - 236615297 (CROSSING Projects P1 and S7), by the German Federal Ministry of Education and Research (BMBF) *iBlockchain Project* (grant nr. 16KIS0902), by the German Federal Ministry of Education and Research and the Hessen State Ministry for Higher Education, Research and the Arts within their joint support of the *National Research Center for Applied Cybersecurity ATHENE*.

REFERENCES

- [1] 2013. Version bytes for BIP32 extended public and private keys. https://electrum.readthedocs.io/en/latest/xpub_version_bytes.html.
- [2] 2014. Ledger Support, Ledger Nano OS. <https://support.ledger.com/hc/en-us/articles/115005297709-Export-your-accounts>.
- [3] 2014. Trezor Wiki, Cryptocurrency standards, Hierarchical deterministic wallets. https://wiki.trezor.io/Cryptocurrency_standards.
- [4] Nabil Alkeilani Alkadri, Poulami Das, Andreas Erwig, Sebastian Faust, Juliane Krämer, Siavash Riahi, and Patrick Struck. 2020. Deterministic Wallets in a Quantum World. In *ACM CCS 20: 27th Conference on Computer and Communications Security*, Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna (Eds.). ACM Press, Virtual Event, USA, 1017–1031. <https://doi.org/10.1145/3372297.3423361>.
- [5] Myrto Arapinis, Andriana Gkaniatsou, Dimitris Karakostas, and Aggelos Kiayias. 2019. A Formal Treatment of Hardware Wallets. In *FC 2019: 23rd International Conference on Financial Cryptography and Data Security (Lecture Notes in Computer Science, Vol. 11598)*, Ian Goldberg and Tyler Moore (Eds.). Springer, Heidelberg, Germany, Frigate Bay, St. Kitts and Nevis, 426–445. https://doi.org/10.1007/978-3-030-32101-7_26.
- [6] Mikhail J. Atallah, Marina Blanton, Nelly Fazio, and Keith B. Frikken. 2009. Dynamic and Efficient Key Management for Access Hierarchies. *ACM Trans. Inf. Syst. Secur.* 12, 3, Article 18 (Jan. 2009), 43 pages. <https://doi.org/10.1145/1455526.1455531>.
- [7] BitcoinExchangeGuide. 2018. CipherTrace Releases Report Exposing Close to \$1 Billion Stolen in Crypto Hacks During 2018. <https://coinexchangeguide.com/ciphertrace-releases-report-exposing-close-to-1-billion-stolen-in-crypto-hacks-during-2018/>.
- [8] Bloomberg. 2018. How to Steal \$500 Million in Cryptocurrency. <http://fortune.com/2018/01/31/coincheck-hack-how/>.
- [9] Dan Boneh, Manu Drijvers, and Gregory Neven. 2018. Compact Multi-signatures for Smaller Blockchains. In *Advances in Cryptology – ASIACRYPT 2018, Part II (Lecture Notes in Computer Science, Vol. 11273)*, Thomas Peyrin and Steven Galbraith (Eds.). Springer, Heidelberg, Germany, Brisbane, Queensland, Australia, 435–464. https://doi.org/10.1007/978-3-030-03329-3_15.
- [10] Joachim Breitner and Nadia Heninger. 2019. Biased Nonce Sense: Lattice Attacks Against Weak ECDSA Signatures in Cryptocurrencies. In *FC 2019: 23rd International Conference on Financial Cryptography and Data Security (Lecture Notes in Computer Science, Vol. 11598)*, Ian Goldberg and Tyler Moore (Eds.). Springer, Heidelberg, Germany, Frigate Bay, St. Kitts and Nevis, 3–20. https://doi.org/10.1007/978-3-030-32101-7_1.
- [11] Michael Brenzel and Christian Rossow. 2018. Identifying Key Leakage of Bitcoin Users. In *Research in Attacks, Intrusions, and Defenses*, Michael Bailey, Thorsten Holz, Manolis Stamatogiannakis, and Sotiris Ioannidis (Eds.). Springer International Publishing, Cham, 623–643.
- [12] Vitalik Buterin. 2013. Deterministic Wallets, Their Advantages and their Understated Flaws. <https://bitcoinmagazine.com/articles/deterministic-wallets-advantages-flaw-1385450276/>.
- [13] Jean-Sébastien Coron. 2002. Optimal Security Proofs for PSS and Other Signature Schemes. In *Advances in Cryptology – EUROCRYPT 2002 (Lecture Notes in Computer Science, Vol. 2332)*, Lars R. Knudsen (Ed.). Springer, Heidelberg, Germany, Amsterdam, The Netherlands, 272–287. https://doi.org/10.1007/3-540-46035-7_18.
- [14] Nicolas T. Courtois, Pinar Emirdag, and Filippo Valsorda. 2014. Private Key Recovery Combination Attacks: On Extreme Fragility of Popular Bitcoin Key Management, Wallet and Cold Storage Solutions in Presence of Poor RNG Events. Cryptology ePrint Archive, Report 2014/848. <https://eprint.iacr.org/2014/848>.
- [15] Poulami Das, Sebastian Faust, and Julian Loss. 2019. A Formal Treatment of Deterministic Wallets. In *ACM CCS 2019: 26th Conference on Computer and Communications Security*, Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz (Eds.). ACM Press, 651–668. <https://doi.org/10.1145/3319535.3354236>.
- [16] Jack Doerner, Yashvanth Kondi, Eysa Lee, and abhi shelat. 2018. Secure Two-party Threshold ECDSA from ECDSA Assumptions. In *2018 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, San Francisco, CA, USA, 980–997. <https://doi.org/10.1109/SP.2018.00036>.
- [17] Manuel Ferschl, Eike Kiltz, and Bertram Poettering. 2016. On the Provable Security of (EC)DSA Signatures. In *ACM CCS 2016: 23rd Conference on Computer and Communications Security*, Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi (Eds.). ACM Press, Vienna, Austria, 1651–1662. <https://doi.org/10.1145/2976749.2978413>.
- [18] Manuel Ferschl, Eike Kiltz, and Bertram Poettering. 2017. On the One-Per-Message Unforgeability of (EC)DSA and Its Variants. In *Theory of Cryptology*, Yael Kalai and Leonid Reyzin (Eds.). Springer International Publishing, Cham, 519–534.
- [19] Marc Fischlin and Nils Fleischhacker. 2013. Limitations of the Meta-reduction Technique: The Case of Schnorr Signatures. In *Advances in Cryptology – EUROCRYPT 2013 (Lecture Notes in Computer Science, Vol. 7881)*, Thomas Johansson and Phong Q. Nguyen (Eds.). Springer, Heidelberg, Germany, Athens, Greece, 444–460. https://doi.org/10.1007/978-3-642-38348-9_27.
- [20] Nils Fleischhacker, Johannes Krupp, Giulio Malavolta, Jonas Schneider, Dominique Schröder, and Mark Simkin. 2016. Efficient Unlinkable Sanitizable Signatures from Signatures with Re-randomizable Keys. In *PKC 2016: 19th International Conference on Theory and Practice of Public Key Cryptography, Part I (Lecture Notes in Computer Science, Vol. 9614)*, Chen-Mou Cheng, Kai-Min Chung, Giuseppe Persiano, and Bo-Yin Yang (Eds.). Springer, Heidelberg, Germany, Taipei, Taiwan, 301–330. https://doi.org/10.1007/978-3-662-49384-7_12.
- [21] Rosario Gennaro, Steven Goldfeder, and Arvind Narayanan. 2016. Threshold-Optimal DSA/ECDSA Signatures and an Application to Bitcoin Wallet Security. In *ACNS 16: 14th International Conference on Applied Cryptography and Network Security (Lecture Notes in Computer Science, Vol. 9696)*, Mark Manulis, Ahmad-Reza Sadeghi, and Steve Schneider (Eds.). Springer, Heidelberg, Germany, Guildford, UK, 156–174. https://doi.org/10.1007/978-3-319-39555-5_9.
- [22] Gus Gutoski and Douglas Stebila. 2015. Hierarchical Deterministic Bitcoin Wallets that Tolerate Key Leakage. In *FC 2015: 19th International Conference on Financial Cryptography and Data Security (Lecture Notes in Computer Science, Vol. 8975)*, Rainer Böhme and Tatsuki Okamoto (Eds.). Springer, Heidelberg, Germany, San Juan, Puerto Rico, 497–504. https://doi.org/10.1007/978-3-662-47854-7_31.
- [23] Saqib A. Kakvi and Eike Kiltz. 2018. Optimal Security Proofs for Full Domain Hash, Revisited. *Journal of Cryptology* 31, 1 (Jan. 2018), 276–306. <https://doi.org/10.1007/s00145-017-9257-9>.
- [24] Eike Kiltz, Daniel Masny, and Jiaxin Pan. 2016. Optimal Security Proofs for Signatures from Identification Schemes. In *Advances in Cryptology – CRYPTO 2016, Part II (Lecture Notes in Computer Science, Vol. 9815)*, Matthew Robshaw and Jonathan Katz (Eds.). Springer, Heidelberg, Germany, Santa Barbara, CA, USA, 33–61. https://doi.org/10.1007/978-3-662-53008-5_2.
- [25] Yashvanth Kondi, Bernardo Magri, Claudio Orlandi, and Omer Shlomovits. 2019. Refresh When You Wake Up: Proactive Threshold Wallets with Offline Devices. Cryptology ePrint Archive, Report 2019/1328. <https://eprint.iacr.org/2019/1328>.
- [26] Yehuda Lindell and Ariel Nof. 2018. Fast Secure Multiparty ECDSA with Practical Distributed Key Generation and Applications to Cryptocurrency Custody. In *ACM CCS 2018: 25th Conference on Computer and Communications Security*, David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang (Eds.). ACM Press, Toronto, ON, Canada, 1837–1854. <https://doi.org/10.1145/3243734.3243788>.
- [27] Adriano Di Luzio, Danilo Francati, and Giuseppe Ateniese. 2020. Arcula: A Secure Hierarchical Deterministic Wallet for Multi-asset Blockchains. In *CANS 20: 19th International Conference on Cryptology and Network Security (Lecture Notes in Computer Science, Vol. 12579)*, Stephan Krenn, Haya Shulman, and Serge Vaudenay (Eds.). Springer, Heidelberg, Germany, Vienna, Austria, 323–343. https://doi.org/10.1007/978-3-030-65411-5_16.
- [28] Antonio Marcedone, Rafael Pass, and abhi shelat. 2019. Minimizing Trust in Hardware Wallets with Two Factor Signatures. In *FC 2019: 23rd International Conference on Financial Cryptography and Data Security (Lecture Notes in Computer Science, Vol. 11598)*, Ian Goldberg and Tyler Moore (Eds.). Springer, Heidelberg, Germany, Frigate Bay, St. Kitts and Nevis, 407–425. https://doi.org/10.1007/978-3-030-32101-7_25.
- [29] Claus-Peter Schnorr. 1990. Efficient Identification and Signatures for Smart Cards. In *Advances in Cryptology – CRYPTO'89 (Lecture Notes in Computer Science, Vol. 435)*, Gilles Brassard (Ed.). Springer, Heidelberg, Germany, Santa Barbara, CA, USA, 239–252. https://doi.org/10.1007/0-387-34805-0_22.
- [30] Victor Shoup. 2004. Sequences of games: a tool for taming complexity in security proofs. Cryptology ePrint Archive, Report 2004/332. <https://ia.cr/2004/332>.
- [31] Rhys Skellern. 2018. Cryptocurrency Hacks: More Than \$2b USD lost between 2011–2018. <https://medium.com/ecom/cryptocurrency-hacks-more-than-2b-usd-lost-between-2011-2018-67054b342219>.
- [32] Mathieu Turuani, Thomas Voegtlin, and Michaël Rusinowitch. 2016. Automated Verification of Electrum Wallet. In *FC 2016 Workshops (Lecture Notes in Computer Science, Vol. 9604)*, Jeremy Clark, Sarah Meiklejohn, Peter Y. A. Ryan, Dan S. Wallach, Michael Brenner, and Kurt Rohloff (Eds.). Springer, Heidelberg, Germany, Christ Church, Barbados, 27–42. https://doi.org/10.1007/978-3-662-53357-4_3.
- [33] Bitcoin Wiki. 2018. BIP32 proposal. https://en.bitcoin.it/wiki/BIP_0032.
- [34] Zongyang Zhang, Yu Chen, Sherman S. M. Chow, Goichiro Hanaoka, Zhenfu Cao, and Yunlei Zhao. 2015. Black-Box Separations of Hash-and-Sign Signatures in the Non-Programmable Random Oracle Model. In *ProvSec 2015: 9th International Conference on Provable Security (Lecture Notes in Computer Science, Vol. 9451)*, Man Ho Au and Atsuko Miyaji (Eds.). Springer, Heidelberg, Germany, Kanazawa, Japan, 435–454. https://doi.org/10.1007/978-3-319-26059-4_24.

A PROOF OF THEOREM 3.3

PROOF. For this proof, we consider an adversary $\mathcal{A}$ playing in the $\text{uf-cma-hrk1}_{\text{REC}[H_1]}$ game relative to a random oracle H_1 . Below, we present a series of games G_0 to G_6 where the following holds.

$$\text{Adv}_{\text{uf-cma-hrk1}_{\text{REC}[H_1]}}^{\mathcal{A}} = \Pr[G_0^{\mathcal{A}} = 1] \leq \Pr[G_6^{\mathcal{A}} = 1] + \frac{q_{H_1}^2}{p}$$

Game G_0 : This game is equivalent to the original game, namely **uf-cma-hrk1** $^{\mathcal{A}}_{\text{REC}[H_1]}$. In particular, a key pair (sk, pk) is sampled as $(sk, pk) \xleftarrow{\$} \text{REC}[H_1].\text{Gen}(\text{par})$. The adversary $\mathcal{A}$ is given pk as the challenge public key and oracle access to Rand , RSign and random oracle H_1 . $\mathcal{A}$ can query Rand to receive a randomness ρ and make a follow-up query to RSign to receive a signature on message m with respect to the rerandomized key $pk' \leftarrow pk + \rho \cdot G$. In particular, $\mathcal{A}$ is allowed to query RSign on every input pair (m, ρ) at most once. Additionally, $\mathcal{A}$ can make direct queries to the random oracle H_1 . Eventually, in order to win the game, $\mathcal{A}$ has to come up with a valid forgery σ^* on a new message m^* with respect to a randomness ρ^* . Since G_0 proceeds as **uf-cma-hrk1** we have that $\Pr[G_0^{\mathcal{A}} = 1] = \Pr[\text{uf-cma-hrk1}^{\mathcal{A}}_{\text{REC}[H_1]} = 1] = \text{Adv}^{\mathcal{A}}_{\text{uf-cma-hrk1}_{\text{REC}[H_1]}}$.

Game G_1 : This game is similar to game G_0 with the following modification. $\mathcal{A}$ is now given a public key $\widetilde{pk}$ instead of pk (which served as the challenge public key in G_0) as the challenge public key. $\widetilde{pk}$ is derived as $\widetilde{pk} \leftarrow pk - \tilde{\rho} \cdot G$ with a freshly sampled randomness $\tilde{\rho} \xleftarrow{\$} \mathcal{R}$. The corresponding secret key is obtained as $\widetilde{sk} = sk - \tilde{\rho}$.

Due to the perfect rerandomizability of keys of the rerandomizable signature scheme REC , pk is indistinguishable from $\widetilde{pk}$. Hence, we have $\Pr[G_0^{\mathcal{A}} = 1] = \Pr[G_1^{\mathcal{A}} = 1]$.

Game G_2 : This game is similar to game G_1 with the following modification in the Rand oracle. An index j is sampled uniformly at random from the set $\{1, \dots, q\}$, where q is an upper bound on the number of queries to the oracle Rand . The game returns $\tilde{\rho}$ at the j^{th} Rand query. For all other queries, ρ is sampled randomly as $\rho \xleftarrow{\$} \mathcal{R}$.

Since both $\tilde{\rho}$ and ρ are sampled randomly from $\mathcal{R}$, the output distribution of the Rand oracle is the same in games G_1 and G_2 . Hence, we have $\Pr[G_2^{\mathcal{A}} = 1] = \Pr[G_1^{\mathcal{A}} = 1]$.

Game G_3 : This game behaves exactly like the game G_2 with the following modifications: First, the game internally maintains a random oracle H_0 (in addition to H_1) in a straightforward manner, by storing a list H_0 of query/response pairs. Second, the game programs the oracle H_1 by maintaining three lists H_1 , H'_1 and Γ , where the first two will be used as possible replies to queries to H_1 , and Γ stores pre-computed signatures. In the beginning of the game, H_1 , H'_1 and Γ are initially set to $\perp$ in each entry. Whenever $\mathcal{A}$ queries a message m to H_1 , the values $H_1[m]$, $H'_1[m]$ and $\Gamma[m]$ are set in one of two ways depending on whether m is prefixed with a public key pk' or not. Here, pk' is a rerandomized form of the public key $\widetilde{pk}$ (i.e., $pk' \leftarrow \widetilde{pk} + \rho \cdot G$ where $\rho \xleftarrow{\$} \text{Rand}$ is a previous answer to any Rand oracle query), where $pk = \widetilde{pk} - \tilde{\rho} \cdot G$ (see Game G_1). Concretely, on query m to H_1 , the lists H_1 , H'_1 and Γ are maintained in the following way:

- If H_1 has already been programmed in a previous query, i.e., $H_1[m] \neq \perp$, return $H_1[m]$.
- Else $H_1[m] = \perp$, then sample uniformly at random $h \xleftarrow{\$} \mathbb{Z}_p$, set $H_1[m] = h$, and proceed as follows:
 - Case 1: m is of the form (pk', m') , where $pk' = \widetilde{pk} + \rho \cdot G = pk + (\rho - \tilde{\rho}) \cdot G$, for $\rho \in \text{RList}$. Derive a signature σ as $\sigma \leftarrow \text{REC}[H_1].\text{Sign}(sk', m')$ for $sk' = sk + \rho = sk + (\rho - \tilde{\rho}) \pmod{p}$ and parse $\sigma := (r, s)$. Then set $H'_1[m] = H_0[m] - r \cdot (\rho - \tilde{\rho}) \pmod{p}$ and $\Gamma[m] = \sigma$. Finally return $H_1[m]$.
 - Case 2: m is not of the form (pk', m') . Set $\Gamma[m] = \epsilon$ and return $H_1[m]$.

In both the cases, the output of H_1 is uniformly distributed from $\mathcal{A}$'s point of view. It follows that $\Pr[G_2^{\mathcal{A}} = 1] = \Pr[G_3^{\mathcal{A}} = 1]$.

Game G_4 : This game proceeds as the previous game with a modification in the Rand oracle. Upon $\mathcal{A}$ querying the Rand oracle, sample ρ as before, then compute the rerandomized public key $pk' \leftarrow pk + \rho \cdot G$ and check if there exists a message m with prefix pk' such that $\Gamma[m] = \epsilon$. In that case, the game aborts.

Claim A.1 Let E_1 be the event that the game G_4 aborts during a Rand query. Then, we have that $\Pr[E_1] \leq \frac{q_{H_1}^2}{p}$.

PROOF. Event E_1 can only occur if $\mathcal{A}$ has queried H_1 on input m with prefix $pk' \leftarrow \widetilde{pk} + \rho \cdot G$ prior to making a query to Rand that returns ρ . Since $\mathcal{A}$ makes at most q_{H_1} queries to H_1 , for each query to Rand that the adversary $\mathcal{A}$ makes, we have that with probability $\frac{q_{H_1}}{p}$ we receive a value ρ such that $pk' \leftarrow \widetilde{pk} + \rho \cdot G$ is a prefix of input m that was earlier made to H_1 . Since there are at most q such queries to Rand by taking the union bound over q_{H_1} we obtain $\Pr[E_1] = \sum_{i=1}^{q_{H_1}} \frac{q_{H_1}}{p} = \frac{q_{H_1}^2}{p}$. ■

From the above, we have that $\Pr[G_3^{\mathcal{A}} = 1] \leq \Pr[G_4^{\mathcal{A}} = 1] + \frac{q_{H_1}^2}{p}$.

Game G_5 : This game is similar to the game G_4 except for a modification in the RSign oracle. Upon $\mathcal{A}$'s query on input (m, ρ) , the game simulates the RSign oracle in the following manner. It computes the rerandomized public key $pk' \leftarrow \widetilde{pk} + \rho \cdot G$ and creates the public key prefixed message $pm \leftarrow (pk', m)$. The signature is implicitly derived via querying the simulated random oracle H_1 (see Game G_3 above) on input the prefixed message pm . This results into $\Gamma[pm] = \sigma = \text{REC}[H_1].\text{Sign}(sk', m)$, which is returned as the response to the signature query.

Observe that all queries to RSign on input the tuple (m, ρ) output the same signature. However, since ECDSA signatures are randomized, the output of RSign should be different with overwhelming probability for each query on the same input tuples. Here, we exploit that $\mathcal{A}$ is allowed to query RSign at most once for the same input pair (m, ρ) . Hence, the output distribution of RSign is identical to the distribution of the RSign oracle in the previous game and it holds that $\Pr[G_4^{\mathcal{A}} = 1] = \Pr[G_5^{\mathcal{A}} = 1]$.

Game G_6 : This game is similar to game G_5 except for the following changes: In the oracles RSign and H_1 the game uses $\text{EC}[H_0].\text{Sign}$ instead of $\text{REC}[H_1].\text{Sign}$ to compute the signatures stored in Γ (and in case of RSign this implicitly happens via H_1). More precisely, when H_1 is queried on $pm = (pk', m')$, where $pk' = \widetilde{pk} + \rho \cdot G = pk + (\rho - \tilde{\rho}) \cdot G$ for $\rho \in \text{RList}$, we derive $\sigma \leftarrow \text{EC}[H_0].\text{Sign}(sk, pm)$, for $sk' = sk + (\rho - \tilde{\rho}) \pmod{p}$. Furthermore, upon H_1 being queried on m , H_1 returns $H'_1[m]$ instead of $H_1[m]$ whenever $\Gamma[m] \neq \perp$ and $\Gamma[m] \neq \epsilon$.

Claim A.2 It holds that $\Pr[G_5^{\mathcal{A}} = 1] = \Pr[G_6^{\mathcal{A}} = 1]$.

PROOF. First, note that in this game, H_1 returns $H_0[m] - r \cdot (\rho - \tilde{\rho})$ on a message m for which a signature is stored in Γ . We have to show now that when H_1 is queried on $pm = (pk', m')$, where $pk' = pk + (\rho - \tilde{\rho}) \cdot G$ and $sk' = sk + (\rho - \tilde{\rho}) \pmod{p}$ for $\rho \in \text{RList}$, we

derive $\sigma \leftarrow \text{EC}[H_0].\text{Sign}(\text{sk}, \text{pm})$ (Game $\mathbf{G}_6$) instead of computing $\sigma \leftarrow \text{REC}[H_1].\text{Sign}(\text{sk}', m')$ (Game $\mathbf{G}_5$).

To this end, we recall Lemma 3.2, which states that if $\sigma = (r, s)$ is a valid signature for $\text{pm} \leftarrow (\text{pk}', m')$ under pk w.r.t. $\text{EC}[H_0]$, it is also a valid signature for m' under $\text{pk}' \leftarrow \text{pk} + (\rho - \tilde{\rho}) \cdot G$ w.r.t. $\text{REC}[H_1]$, if it holds that $H_1(\text{pm}) = H_0(\text{pm}) - r \cdot (\rho - \tilde{\rho}) \pmod{p}$. Note that we replaced the $\text{REC}[H_1].\text{Sign}$ procedure call on a message m' in $\mathbf{G}_5$ by a $\text{EC}[H_0].\text{Sign}$ procedure call on a prefixed message $\text{pm} \leftarrow (\text{pk}', m')$, where $\text{pk}' = \text{pk} + (\rho - \tilde{\rho}) \cdot G$. It remains to show that the condition $H_1(\text{pm}) = H_0(\text{pm}) - r \cdot (\rho - \tilde{\rho}) \pmod{p}$ holds. But since $H'_1[\text{pm}] = H_0[\text{pm}] - r \cdot (\rho - \tilde{\rho}) \pmod{p}$ is programmed accordingly (latest when RSign is queried), this follows directly. ■

Combining results from $\mathbf{G}_0$ to $\mathbf{G}_6$, we have that

$$\Pr[\mathbf{G}_0^{\mathcal{A}} = 1] \leq \Pr[\mathbf{G}_6^{\mathcal{A}} = 1] + \frac{q_{H_1}^2}{p}. \quad (1)$$

Reduction to uf-cma1 security. Having shown that the original $\text{uf-cma1-hrk1}_{\text{REC}[H_1]}^{\mathcal{A}}$ game is indistinguishable from game $\mathbf{G}_6$, it remains to show that an adversary $\mathcal{A}$ winning in game $\mathbf{G}_6$ can be turned into an adversary C that wins $\text{uf-cma1}_{\text{EC}[H_0]}^C$ game with related success probability. To this end, we construct C that runs in the game $\text{uf-cma1}_{\text{EC}[H_0]}^C$ and simulates to $\mathcal{A}$ game $\mathbf{G}_6$. Thus, C proceeds as game $\mathbf{G}_6$ and leverages oracle access to its own signing oracle (with respect to its challenge public key) in the following way:

- (1) On input the challenge public key pk_C from $\text{uf-cma1}_{\text{EC}[H_0]}^C$, the adversary C sets pk to pk_C . Note that this implicitly sets the challenge public key in C 's simulation of $\mathbf{G}_6$ to $\text{pk} = \text{pk}_C - \tilde{\rho} \cdot G$. Hence, C runs $\mathcal{A}$ on input pk .
- (2) In case $\mathcal{A}$ returns a forgery (m^*, σ^*, ρ^*) with $\rho^* \neq \tilde{\rho}$, C aborts. C perfectly simulates $\mathbf{G}_6$ for $\mathcal{A}$ except in case where it aborts. Moreover, note that in case there is no abort, we have that

$$\text{pk}^* = \tilde{\text{pk}} + \rho^* \cdot G = \text{pk}_C - \tilde{\rho} \cdot G + \tilde{\rho} \cdot G = \text{pk}_C.$$

From the above programming strategy, we conclude that for $\mathcal{A}$'s queries to H_1 that are prefixed with pk^* , the oracles H_0 and H_1 are identical. It remains to calculate the success probability of C in winning the $\text{uf-cma1}_{\text{EC}[H_0]}^C$ game in case $\mathcal{A}$ returns a valid forgery.

Claim A.3 Let E_2 be the event that $\mathcal{A}$ outputs (m^*, σ^*, ρ^*) s.t. (pm^*, σ^*) constitutes a valid forgery in game $\text{uf-cma1}_{\text{EC}[H_0]}^C$. Then, we have that $\Pr[E_2 | \mathbf{G}_6^{\mathcal{A}} = 1] \geq \frac{1}{q}$, where q is the number of queries to the Rand oracle.

PROOF. In order to prove this claim, we need to show that with probability $\frac{1}{q}$ it must hold that (1) (pm^*, σ^*) is a valid forgery in game $\text{uf-cma1}_{\text{EC}[H_0]}^C$ under public key pk_C and (2) the Sign oracle of the $\text{uf-cma1}_{\text{EC}[H_0]}^C$ game has not been queried on input pm^* . First, note that if σ^* is a valid signature for message (pk^*, m^*) under the public key pk^* relative to $\text{REC}[H_1]$, then σ^* is also a valid signature on pm^* under public key $\text{pk}_C = \text{pk}^*$ relative to $\text{EC}[H_0]$, as H_0 and H_1 are identical for messages prefixed with pk^* . Since there are at most q possible values of ρ^* and C chooses one of

them uniformly at random, the probability that C 's guess is correct is at least $\frac{1}{q}$. Note that from the adversary's perspective, the public key generated at index j is no different than other public keys. Second, since (m^*, σ^*, ρ^*) is a valid forgery in $\text{uf-cma1-hrk1}_{\text{REC}[H_1]}^{\mathcal{A}}$, $\mathcal{A}$ has not previously queried the RSign oracle on input (m^*, ρ^*) . Correspondingly, the Sign oracle of the $\text{uf-cma1}_{\text{EC}[H_0]}$ game has also not been queried on message pm^* and hence, (pm^*, σ^*) is a valid forgery in $\text{uf-cma1}_{\text{EC}[H_0]}$. ■

From Eq. 1 we get the following.

$$\begin{aligned} \text{Adv}_{\text{uf-cma1-hrk1}_{\text{REC}[H_1]}^{\mathcal{A}}} &= \Pr[\mathbf{G}_0^{\mathcal{A}} = 1] \leq \Pr[\mathbf{G}_6^{\mathcal{A}} = 1] + \frac{q_{H_1}^2}{p} \\ \text{or, } \Pr[\mathbf{G}_6^{\mathcal{A}} = 1] &\geq \text{Adv}_{\text{uf-cma1-hrk1}_{\text{REC}[H_1]}^{\mathcal{A}}} - \frac{q_{H_1}^2}{p} \end{aligned}$$

Since C can use a valid forgery by $\mathcal{A}$ in its own game whenever E_2 occurs,

$$\begin{aligned} \text{Adv}_{\text{uf-cma1}_{\text{EC}[H_0]}^C} &\geq \Pr[\mathbf{G}_6^{\mathcal{A}} = 1] \cdot \Pr[E_2 | \mathbf{G}_6^{\mathcal{A}} = 1] = \Pr[\mathbf{G}_6^{\mathcal{A}} = 1] \cdot \frac{1}{q} \\ &\geq \left(\text{Adv}_{\text{uf-cma1-hrk1}_{\text{REC}[H_1]}^{\mathcal{A}}} - \frac{q_{H_1}^2}{p} \right) \cdot \frac{1}{q} \end{aligned}$$

B UNLINKABILITY PROOF OF GENERIC CONSTRUCTION

Theorem B.1 Let $\text{HDWal}[\text{RSig}]$ be the construction defined in Figure 5. Then for any adversary $\mathcal{A}$ playing in game $\text{unl}_{\text{HDWal}[\text{RSig}]}^{\mathcal{A}}$ there exists an adversary $\mathcal{A}_1$ that plays in the game $\text{uf-cma1-hrk1}_{\text{RSig}}$ such that

$$\text{Adv}_{\text{unl}_{\text{HDWal}[\text{RSig}]}^{\mathcal{A}}} \leq \frac{q_H(q_C + 1)}{2^\kappa} + \text{Adv}_{\text{uf-cma1-hrk1}_{\text{RSig}}}^{\mathcal{A}_1}$$

where q_H and q_C are the number of random oracle and child creation queries from $\mathcal{A}$, respectively.

PROOF. Consider the $\text{unl}_{\text{HDWal}[\text{RSig}]}^{\mathcal{A}}$ game for an adversary $\mathcal{A}$. In the beginning, the challenger generates a fresh master key pair and chaincode

$$(\text{msk}_{\text{nh},0,0}, \text{mpk}_{\text{nh},0,0}, \text{ch}_{0,0}) \leftarrow \text{HDWal}[\text{RSig}].\text{Setup}(\text{par})$$

and runs $\mathcal{A}$ on inputs the security parameter and the master public key $\text{mpk}_{\text{nh},0,0}$. During the output phase of the $\text{unl}_{\text{HDWal}[\text{RSig}]}^{\mathcal{A}}$ game, $\mathcal{A}$ outputs a tuple $(\text{addr}_{i,s}, \text{e}_{i+1}^{s,t}, c)$, where $\text{e}_{i+1}^{s,t}$ is the edge from the node with address $\text{addr}_{i,s}$ to the challenge node with address $\text{addr}_{i+1,t}$ and c indicates if $\text{addr}_{i+1,t}$ is a hardened or non-hardened node. In case $\text{addr}_{i,s}$ is a hardened node, the game aborts and hence we have that the adversary's advantage is 0, i.e., $\text{Adv}_{\text{unl}_{\text{HDWal}[\text{RSig}]}^{\mathcal{A}}} = 0$. Likewise, if $\mathcal{A}$ has previously queried the CHLeak0 oracle on address $\text{addr}_{i,s}$ or any of its prefix addresses, i.e., $\text{CH}[\text{addr}_{i,s}^j] = 1$ for any $j < i$, and if the challenge node is non-hardened (i.e., $c = \text{nh}$) then the game aborts and we have that $\text{Adv}_{\text{unl}_{\text{HDWal}[\text{RSig}]}^{\mathcal{A}}} = 0$.

Let $\text{pk}_{\text{nh},i+1,t}$ and $\text{pk}_{\text{nh},1,t}$ denote the challenge public keys in case $\mathcal{A}$ challenges a non-hardened node (i.e., $c = \text{nh}$) with

address $\text{addr}_{i+1,t}$. Further, let $(pk_{j,\cdot}, ch_{j,\cdot})$ for $1 \leq j \leq i$ denote the public key and chaincode pair of all nodes in the prefix address of $\text{addr}_{i+1,t}$. Recall that the non-hardened public keys are computed as follows:

$$\begin{aligned} (\omega, ch_{j+1,t}) &\leftarrow H(pk_{j,s}, ch_{j,s}, e_{j+1}^{s,t}), \\ pk_{j+1,t} &\leftarrow \text{RSig.RandPK}(pk_{j,s}; \omega) \end{aligned}$$

According to the (perfect) rerandomizability of keys property (cf. Def 2.2) the public keys derived via the RSig.RandPK algorithm are identically distributed to freshly generated keys from $\mathcal{A}$'s view as long as ω is uniformly random. Therefore, the challenge public keys $pk_{nh,i+1,t}$ and $pk_{nh,1,t}$ are identically distributed from $\mathcal{A}$'s point of view as long as $\mathcal{A}$ has not previously queried the random oracle H on input $(pk_{j,\cdot}, ch_{j,\cdot}, e_{j+1}^{s,t})$. If $\mathcal{A}$ makes one of the aforementioned queries, it can recursively compute the public key of the challenge node, thereby trivially winning the $\text{unl}_{\text{HDWal}[\text{RSig}]}^{\mathcal{A}}$ game. By assumption, $\mathcal{A}$ makes at most q_C queries to the child creation oracles. Therefore, there are at most $q_C + 1$ potential chaincodes that $\mathcal{A}$ can guess correctly and query the random oracle on. For each of these, the probability of correctly guessing it is $\frac{1}{2^\kappa}$ and thereby the probability of correctly guessing any of the chaincodes is at most $\frac{q_C+1}{2^\kappa}$ during any particular random oracle query. Since $\mathcal{A}$ makes at most q_H calls to H , the overall probability of querying the random oracle on an input as above is $\frac{q_H(q_C+1)}{2^\kappa}$.

It remains to show $\mathcal{A}$'s probability of winning the $\text{unl}_{\text{HDWal}[\text{RSig}]}^{\mathcal{A}}$ game in case the adversary challenges a hardened node with address $\text{addr}_{i+1,t}$. In this case, let $pk_{h,i+1,t}$ and $pk_{h,1,t}$ denote the challenge public keys and let $(pk_{j,\cdot}, ch_{j,\cdot})$ for $1 \leq j \leq i$ denote the public key and chaincode pair of all nodes in the prefix address of $\text{addr}_{i+1,t}$.

$\mathcal{A}$ is allowed to query the CHLeakO oracle for parent nodes, thereby eliminating the need to correctly guess a relevant chaincode. Recall that hardened public keys are derived as follows:

$$\begin{aligned} (\omega, ch_{j+1,t}) &\leftarrow H(sk_{j,s}, ch_{j,s}, e_{j+1}^{s,t}) \\ pk_{j+1,t} &\leftarrow \text{RSig.RandPK}(pk_{j,s}; \omega) \end{aligned}$$

Hence, having access to the CHLeakO oracle does not reveal all required inputs to the random oracle, i.e., the secret key of the parent node is still unknown to the adversary. As such, according to the (perfect) rerandomizability of keys property (cf. Def 2.2), $\mathcal{A}$ can distinguish $pk_{h,i+1,t}$ from $pk_{h,1,t}$ only if it is able to compute the secret key of one of the challenge nodes' parents. Let E be the event that $\mathcal{A}$ can compute a secret key $sk_{j,\cdot}$ that corresponds to any of the public keys $pk_{j,\cdot}$ and calls the random oracle on input $(sk_{j,\cdot}, \cdot, \cdot)$. Then we can upper bound the probability that event E occurs as follows:

Claim B.2 There exists an algorithm $\mathcal{A}_1$ such that

$$\text{Adv}_{\text{uf-cma-hrk1RSig}}^{\mathcal{A}_1} \geq \Pr[E].$$

PROOF. The proof of this claim corresponds to the proof of claim C.1 in Appendix C. ■

Therefore, the adversary's advantage in case of a hardened challenge node can be upper bounded by $\text{Adv}_{\text{uf-cma-hrk1RSig}}^{\mathcal{A}_1}$ and $\mathcal{A}$'s

overall advantage in game $\text{unl}_{\text{HDWal}[\text{RSig}]}^{\mathcal{A}}$ can be upper bounded by $\text{Adv}_{\text{unl}_{\text{HDWal}[\text{RSig}]}^{\mathcal{A}}}^{\mathcal{A}} \leq \frac{q_H(q_C+1)}{2^\kappa} + \text{Adv}_{\text{uf-cma-hrk1RSig}}^{\mathcal{A}_1}$. ■

Indistinguishability of Hardened nodes. Recall that in our construction $\text{HDWal}[\text{RSig}]$, a hardened key pair $(sk_{h,(i+1),t}, pk_{h,(i+1),t})$ is derived via SKDer_H and PKDer_H as follows:

$$\begin{aligned} (\omega, ch_{(i+1),t}) &\leftarrow H(sk_{nh,i,s}, ch_{i,s}, e_{i+1}^{s,t}) \\ sk_{h,(i+1),t} &\leftarrow \text{RSig.RandSK}(sk_{nh,i,s}; \omega) \\ pk_{h,(i+1),t} &\leftarrow \text{RSig.RandPK}(pk_{nh,i,s}; \omega) \end{aligned}$$

Due to the key rerandomizability property of the underlying signature scheme RSig , $\mathcal{A}$ can only distinguish $(sk_{h,(i+1),t}, pk_{h,(i+1),t})$ from a fresh key pair if it can distinguish ω from random. Since we model H as a random oracle, this happens only if $\mathcal{A}$ has previously queried H on the same input, i.e., $(sk_{nh,i,s}, St_{i,s}, e_{i+1}^{s,t})$. Since our model excludes secret key leakage of non-hardened nodes, the adversary cannot distinguish the output of H from a random value except if it correctly guesses $sk_{nh,i,s}$ or any parent secret key that $sk_{nh,i,s}$ has been (directly or indirectly) derived from.

C PROOF OF THEOREM 5.3

PROOF. Before we give the full formal proof of Theorem 5.3, we first provide a high level overview of the proof. We show that the generic construction $\text{HDWal}[\text{RSig}]$ is one-per message unforgeable w.r.t. game $\text{wufcma1}_{\text{HDWal}[\text{RSig}]}$, if the signature scheme with rerandomizable keys RSig is one-per message unforgeable w.r.t. the game $\text{uf-cma-hrk1}_{\text{RSig}}$. The main idea behind the proof is as follows. First, the adversary C in game $\text{uf-cma-hrk1}_{\text{RSig}}$ receives a public key pk_C . It chooses a random chaincode and uses it to derive a key mpk , which it embeds mpk as the master public key for $\mathcal{A}$ in game $\text{wufcma1}_{\text{HDWal}[\text{RSig}]}$. Note that these changes cannot be detected by $\mathcal{A}$ due to the rerandomizability of keys and the transitivity property of the RSig scheme (see Definitions 2.2 and 5.1). Second, C attempts to predict, for each hardened node in the tree, with a certain probability if this node will get corrupted by $\mathcal{A}$ throughout the game. For these nodes, C generates a fresh key pair independently of pk_C . For the other hardened nodes, C derives non-hardened public keys (instead of hardened public keys) from pk_C . It is crucial here that the non-hardened public keys are derived from pk_C instead of from the parent public key, since pk_C is not known to the adversary and therefore $\mathcal{A}$ is not able to distinguish the non-hardened public key from a hardened public key. We show that this guessing introduces a polynomial loss in the number of corrupted hardened nodes for C 's advantage to win game $\text{uf-cma-hrk1}_{\text{RSig}}$ game but that C still wins the game with non-negligible probability.

We now provide the formal proof via a series of games G_0 to G_6 .

Game G_0 : This is the regular $\text{wufcma1}_{\text{HDWal}[\text{RSig}]}$ game at the beginning of which a key pair (pk, sk) is generated and the adversary $\mathcal{A}$ is given as input pk and oracle access to the following oracles: $H\text{ChildO}$, $NH\text{ChildO}$, $H\text{DSignO}$, $CH\text{LeakO}$ and $SK\text{LeakO}$ oracles and a random oracle H . The random oracle H is internally programmed in a straight forward manner, by maintaining a list

H . In particular, on input s , if $H[s] \neq \perp$, then return $H[s]$. Otherwise, sample a fresh randomness $\rho \xleftarrow{\$} \mathcal{R}$ and a fresh value as $\psi \xleftarrow{\$} \{0, 1\}^\kappa$ and set $(\rho, \psi) =: H[s]$ and return $H[s]$. In addition, the game keeps a list R in which it stores the randomness used to derive the keys at position s and level i at entry $R[i, s]$. We have that $\text{Adv}_{\text{wufcma1HDWal[RSig]}^{\mathcal{A}}} = \Pr[\text{wufcma1}_{\text{HDWal[RSig]}^{\mathcal{A}}} = 1] = \Pr[\mathbf{G}_0^{\mathcal{A}} = 1]$.

Game $\mathbf{G}_1$: Upon generating the key pair (pk, sk) , the game chooses a fresh chaincode $ch_{0,0} \xleftarrow{\$} \{0, 1\}^\kappa$ and fresh randomness $\rho \xleftarrow{\$} \mathcal{R}$. Then it derives the root public key for the $\text{wufcma1}_{\text{HDWal[RSig]}}$ game as $mpk_{0,0} \xleftarrow{\$} \text{RSig.RandPK}(pk; \rho)$, stores ρ in a list as $R[0, 0] = \rho$. The game sends $ch_{0,0}$ and $mpk_{0,0}$ to $\mathcal{A}$.

Since the randomness ρ is chosen uniformly at random from $\mathcal{R}$, the rerandomizability of keys property of the signature scheme RSig holds. This implies that the distributions of $(\cdot, mpk_{0,0})$ and $(\cdot, mpk'_{0,0}) \xleftarrow{\$} \text{RSig.Gen}(\text{par})$ are identical. Therefore, it holds that $\Pr[\mathbf{G}_1^{\mathcal{A}} = 1] = \Pr[\mathbf{G}_0^{\mathcal{A}} = 1]$.

Game $\mathbf{G}_2$: This game behaves like $\mathbf{G}_1$ with a modification in the NHChild0 oracle. Upon an oracle query on input $(\text{addr}_{i,s}, e_{i+1}^{s,t})$ the NHChild0 oracle executes $\text{PKDer}_{\text{NH}}(pk_{i,s}, ch_{i,s}, \text{addr}_{i,s}, e_{i+1}^{s,t})$ and creates the public key $pk_{nh,i+1,t}$ at level $i+1$ and position t as $pk_{nh,i+1,t} \leftarrow \text{RandPK}(pk; \omega \odot R[i, s])$, i.e., the public key $pk_{nh,i+1,t}$ is derived directly from pk with randomness $\omega \odot R[i, s]$, where $(\omega, \cdot) \leftarrow H(pk_{i,s}, ch_{i,s}, e_{i+1}^{s,t})$. The game then sets the list $R[i+1, t] = \omega \odot R[i, s]$. If any of the values $(pk_{i,s}, ch_{i,s}, \text{addr}_{i,s}, R[i, s])$ is not defined yet, the game recursively derives the path from the root node up to $(pk_{i,s}, \text{addr}_{i,s})$ and updates the list up to $R[i, s]$.

Note that $\text{RandPK}(pk; \omega \odot R[i, s])$ and $\text{RandPK}(pk_{nh,i,s}; \omega)$ derive the same key $pk_{nh,i+1,t}$, due to the transitive property of rerandomizable keys. Since ω and $R[i, s]$ are uniformly at random from $\mathcal{R}$, we have that $\Pr[\mathbf{G}_2^{\mathcal{A}} = 1] = \Pr[\mathbf{G}_1^{\mathcal{A}} = 1]$.

Game $\mathbf{G}_3$: This game proceeds similarly to the previous game with a modification in the random oracle. The game aborts upon the adversary querying the random oracle on input $(sk_{nh,i,s}, \cdot, \cdot)$ where $sk_{nh,i,s}$ is either a non-hardened secret key that corresponds to a public key $pk_{nh,i,s}$ previously output by the NHChild0 oracle or $sk_{nh,i,s}$ is the master secret key $msk_{0,0}$ corresponding to $mpk_{0,0}$.

Claim C.1 Let ϵ be the probability that game $\mathbf{G}_3$ aborts during a random oracle query. Then there exists an algorithm C_1 playing in game $\text{uf-cma-hrk1}_{\text{RSig}}$ such that $\text{Adv}_{\text{uf-cma-hrk1}_{\text{RSig}}}^{C_1} \geq \epsilon$.

PROOF. We prove this claim by providing a reduction to the uf-cma-hrk1 security of RSig . More concretely, we show that there exists an algorithm C_1 with $\text{Adv}_{\text{uf-cma-hrk1}_{\text{RSig}}}^{C_1} \geq \epsilon$ assuming C_1 has access to an adversary $\mathcal{A}$ that causes $\mathbf{G}_3$ to abort with probability ϵ . Initially, C_1 receives as input a public key pk from the $\text{uf-cma-hrk1}_{\text{RSig}}$ game and chooses at random a chaincode $ch \xleftarrow{\$} \{0, 1\}^\kappa$. From pk and ch , C_1 can honestly simulate the NHChild0 and CHLeak0 oracles to $\mathcal{A}$. The simulation of the random oracle H works as described in $\mathbf{G}_3$ with the exception that instead of sampling the randomness $\rho \xleftarrow{\$} \mathcal{R}$ uniformly at random from $\mathcal{R}$,

C_1 calls the Rand oracle in game $\text{uf-cma-hrk1}_{\text{RSig}}$ to obtain the randomness ρ . A query from $\mathcal{A}$ to the HDSig0 oracle on input $(m, \text{addr}_{\cdot, \cdot})$ is forwarded to the RSig oracle on input m and the randomness corresponding to $\text{addr}_{\cdot, \cdot}$ of the $\text{uf-cma-hrk1}_{\text{RSig}}^{C_1}$ game.

For a HChild0 oracle query on input $(\text{addr}_{i,s}, e_{i+1}^{s,t})$, C_1 chooses a fresh key pair (independently of pk) $(sk', pk') \xleftarrow{\$} \text{RSig.Gen}(\text{par})$, assigns $(sk_{h,i+1,t}, pk_{h,i+1,t}) := (sk', pk')$ and returns $pk_{h,i+1,t}$. The SKLeak0 oracle is then simulated by returning $sk_{h,i+1,t}$ on input $\text{addr}_{i+1,t}$. The simulation of the HChild0 and HDSig0 oracles cannot be distinguished by $\mathcal{A}$ from the oracles in $\mathbf{G}_3$ due to the rerandomizability of keys property of RSig . The only way in which $\mathcal{A}$ could detect the difference between $\mathbf{G}_3$ and the reduction provided by C_1 would be if the following event occurs. $\mathcal{A}$ makes a random oracle query of the form $(sk_{nh,i,s}, \cdot, \cdot)$ where $sk_{nh,i,s}$ is either a non-hardened secret key that corresponds to a public key $pk_{nh,i,s}$ previously output by the NHChild0 oracle or $sk_{nh,i,s}$ is the secret key corresponding to pk (if $sk_{nh,i,s}$ belongs to a public key $pk_{nh,i,s}$ can be efficiently checked via the function $\text{ToPubKey}(sk_{nh,i,s})$). By Claim C.1, this event happens with probability ϵ . However, when this event occurs, C_1 learns the secret key $sk_{nh,i,s}$ which it can use to compute the secret key sk of the $\text{uf-cma-hrk1}_{\text{RSig}}$ game. This is due to the transitivity and invertible rerandomization property of RSig . C_1 can then use sk to create a valid forgery in the $\text{uf-cma-hrk1}_{\text{RSig}}^{C_1}$ game. Therefore, we have that

$$\text{Adv}_{\text{uf-cma-hrk1}_{\text{RSig}}}^{C_1} \geq \epsilon. \quad \blacksquare$$

It follows that $\Pr[\mathbf{G}_2^{\mathcal{A}}] \leq \Pr[\mathbf{G}_3^{\mathcal{A}}] + \epsilon$.

Game $\mathbf{G}_4$: This game works like the previous game with a modification to the HChild0 oracle which works as follows. Let q_{sk} be the number of hardened nodes that $\mathcal{A}$ corrupts via the SKLeak0 oracle. Upon $\mathcal{A}$ querying the HChild0 oracle, with probability $\frac{1}{q_{sk}+1}$, the address of this node is added to a list L . Let Bad define the event that a node corresponding to an address in L is corrupted.

Since the change in this game is only syntactical, $\mathcal{A}$'s winning probability is not affected by whether Bad occurs. It follows that $\Pr[\mathbf{G}_3^{\mathcal{A}}] = \Pr[\mathbf{G}_4^{\mathcal{A}}]$.

Game $\mathbf{G}_5$: This game works like the previous game with the only difference that $\mathbf{G}_5$ aborts in case event Bad occurs.

Lemma C.2 $\Pr[\mathbf{G}_4^{\mathcal{A}} = 1] \leq \Pr[\mathbf{G}_5^{\mathcal{A}} = 1] \cdot e$.

PROOF. $\mathcal{A}$ can distinguish $\mathbf{G}_5$ from the previous game if the game aborts i.e., when the event Bad happens. This event happens for each SKLeak0 query, independently, with probability $\frac{1}{q_{sk}+1}$. With probability $(1 - \frac{1}{q_{sk}+1})$, a SKLeak0 oracle query does not lead to an abort. Hence, the overall probability with which the game does not abort on any SKLeak0 oracle query can be lower bounded by $(1 - \frac{1}{q_{sk}+1})^{q_{sk}} \geq e^{-1}$, i.e., Bad occurs with probability at most $1 - e^{-1}$. As we have argued that Bad occurs in $\mathbf{G}_4$ independently of the event $\mathbf{G}_4 = 1$, we have that $\Pr[\mathbf{G}_4^{\mathcal{A}} = 1] = \Pr[\mathbf{G}_5^{\mathcal{A}} = 1] \cdot 1 / \Pr[\neg \text{Bad}] \leq \Pr[\mathbf{G}_5^{\mathcal{A}} = 1] \cdot e$. $\blacksquare$

Game G_6 : This game works like the previous game with a modification to the $HChild0$ oracle which works as follows. For the nodes that are chosen to be added to the list L , the game derives the public key of that node as a public key of a non-hardened node. The rest of the hardened nodes are generated as $(sk, pk) \xleftarrow{\$} \text{RSig.Gen}(\text{par})$ and assigned $(sk_{h,i+1,t}, pk_{h,i+1,t}) := (sk, pk)$.

Lemma C.3 $\Pr[G_5^{\mathcal{A}} = 1] = \Pr[G_6^{\mathcal{A}} = 1]$.

PROOF. $\mathcal{A}$ can distinguish G_6 from the previous game if it corrupts a hardened node which is simulated as a non-hardened node i.e., one of the nodes in the list L . The only other way for $\mathcal{A}$ to distinguish these two games would be if $\mathcal{A}$ was able to query the random oracle on input the secret key of a non-hardened node as this would allow to recursively compute the secret key of the corresponding child hardened node. This case is, however, has already been excluded in G_3 . As explained in game G_5 , upon $\mathcal{A}$ making a corruption query for a node in list L , the game aborts. Therefore, the adversary cannot distinguish this game from the previous game. ■

By the transition from game G_0 to game G_6 , we get that

$$\begin{aligned} \text{Adv}_{\text{wufcma1}_{\text{HDWal}}[\text{RSig}]}^{\mathcal{A}} &= \Pr[G_0^{\mathcal{A}} = 1] \\ &\leq (\Pr[G_6^{\mathcal{A}} = 1] \cdot e) + \epsilon \\ \text{or, } \Pr[G_6^{\mathcal{A}} = 1] &\geq \frac{1}{e} \cdot \text{Adv}_{\text{wufcma1}_{\text{HDWal}}[\text{RSig}]}^{\mathcal{A}} - \frac{1}{e} \cdot \epsilon \end{aligned}$$

Reduction to uf-cma-hrk security. Having shown that the transition from game $\text{wufcma1}_{\text{HDWal}}^{\mathcal{A}}[\text{RSig}]$ to the game G_6 is indistinguishable, it remains to show that there exists a challenger C_2 that simulates G_5 and uses $\mathcal{A}$ to win its own game $\text{uf-cma-hrk1}_{\text{RSig}}^{\mathcal{A}}$. The challenger code is same as G_6 with the following changes: (1) The sampling of $\rho \xleftarrow{\$} \mathcal{R}$ within the programming of H is replaced by a call to the oracle Rand (2) pk is replaced by the challenge public key pk_{C_2} from the underlying game $\text{uf-cma-hrk1}_{\text{RSig}}^{C_2}$. Since the above changes are trivially indistinguishable to $\mathcal{A}$, we move on to analyze C_2 's probability to win the $\text{uf-cma-hrk1}_{\text{RSig}}^{C_2}$ game using the forgery of $\mathcal{A}$. There are two possibilities for $\mathcal{A}$; either to output a forgery for a non-hardened node or for a hardened node. We analyze each case separately and show that for both cases our simulator can win its game with non-negligible probability.

- **Adversary outputs a forgery for a non-hardened node:** If the adversary provides a forgery for a non-hardened node, C_2 can always use this forgery to win the $\text{uf-cma-hrk1}_{\text{RSig}}^{C_2}$ game. Therefore, the overall probability of C_2 winning the game in case of $\mathcal{A}$ generating a forgery for a non-hardened node is:

$$\begin{aligned} \text{Adv}_{\text{uf-cma-hrk1}_{\text{RSig}}}^{C_2} &\geq \Pr[G_6^{\mathcal{A}} = 1] \\ &\geq \frac{1}{e} \cdot \text{Adv}_{\text{wufcma1}_{\text{HDWal}}[\text{RSig}]}^{\mathcal{A}} - \frac{\epsilon}{e} \end{aligned}$$

- **Adversary outputs a forgery for a hardened node:** We now compute the probability that the game aborts in case the adversary generates a forgery for a hardened node.

Let i^* be the index of the hardened node for which the adversary outputs a forgery. In this case C_2 needs to abort if i^* was sampled randomly. Recall, the probability that i^* has been sampled at random is $1 - \frac{1}{q_{sk}+1}$. Therefore, the overall probability of the simulator winning the game in case of $\mathcal{A}$ generating a forgery for a hardened node is:

$$\begin{aligned} \text{Adv}_{\text{uf-cma-hrk1}_{\text{RSig}}}^{C_2} &\geq \Pr[G_6^{\mathcal{A}} = 1] \cdot \frac{1}{q_{sk} + 1} \\ &\geq \left(\frac{1}{e} \cdot \text{Adv}_{\text{wufcma1}_{\text{HDWal}}[\text{RSig}]}^{\mathcal{A}} - \frac{\epsilon}{e} \right) \cdot \frac{1}{q_{sk} + 1} \end{aligned}$$

We can now compose a challenger C from the challengers C_1 of Claim C.1 and C_2 , such that C uses adversary $\mathcal{A}$ to win in its game $\text{uf-cma-hrk1}_{\text{RSig}}^C$. C executes either of C_1 and C_2 with probability $\frac{1}{2}$. In order to compute C 's advantage $\text{Adv}_{\text{uf-cma-hrk1}_{\text{RSig}}}^C$, we distinguish the following two cases:

- **Case $\epsilon \geq \frac{1}{2} \text{Adv}_{\text{wufcma1}_{\text{HDWal}}}^{\mathcal{A}}$:** In this case, we have by claim C.1 that

$$\text{Adv}_{\text{uf-cma-hrk1}_{\text{RSig}}}^{C_1} \geq \epsilon \geq \frac{1}{2} \text{Adv}_{\text{wufcma1}_{\text{HDWal}}}^{\mathcal{A}}.$$

Therefore we can lower bound C 's advantage by

$$\text{Adv}_{\text{uf-cma-hrk1}_{\text{RSig}}}^C \geq \frac{1}{2} \text{Adv}_{\text{uf-cma-hrk1}_{\text{RSig}}}^{C_1} \geq \frac{\text{Adv}_{\text{wufcma1}_{\text{HDWal}}}^{\mathcal{A}}}{4}.$$

- **Case $\epsilon < \frac{1}{2} \text{Adv}_{\text{wufcma1}_{\text{HDWal}}}^{\mathcal{A}}$:** In this case, we can lower bound C_2 's advantage by

$$\begin{aligned} \text{Adv}_{\text{uf-cma-hrk1}_{\text{RSig}}}^{C_2} &\geq \left(\text{Adv}_{\text{wufcma1}_{\text{HDWal}}[\text{RSig}]}^{\mathcal{A}} \cdot \frac{1}{e} - \frac{\epsilon}{e} \right) \cdot \frac{1}{q_{sk} + 1} \\ &\geq \left(\text{Adv}_{\text{wufcma1}_{\text{HDWal}}[\text{RSig}]}^{\mathcal{A}} \cdot \frac{1}{e} - \frac{1}{2e} \cdot \text{Adv}_{\text{wufcma1}_{\text{HDWal}}[\text{RSig}]}^{\mathcal{A}} \right) \cdot \frac{1}{q_{sk} + 1} \\ &= \frac{1}{2e(q_{sk} + 1)} \cdot \text{Adv}_{\text{wufcma1}_{\text{HDWal}}[\text{RSig}]}^{\mathcal{A}} \end{aligned}$$

Hence, C 's overall advantage can be lower bounded by

$$\begin{aligned} \text{Adv}_{\text{uf-cma-hrk1}_{\text{RSig}}}^C &\geq \min \left(\frac{1}{2} \text{Adv}_{\text{uf-cma-hrk1}_{\text{RSig}}}^{C_1}, \frac{1}{2} \text{Adv}_{\text{uf-cma-hrk1}_{\text{RSig}}}^{C_2} \right) \\ &\geq \frac{1}{4e(q_{sk} + 1)} \cdot \text{Adv}_{\text{wufcma1}_{\text{HDWal}}[\text{RSig}]}^{\mathcal{A}}. \end{aligned}$$

D IMPOSSIBILITY OF A TIGHTER BOUND ■

In this section, we first recall the security notion of unforgeability under rerandomized keys for signature schemes with rerandomizable keys as introduced in [20]. This notion is stronger than the notion of unforgeability under *honestly* rerandomized keys in the sense that an adversary is not restricted to randomness chosen uniformly at random from the randomness space $\mathcal{R}$ for the key rerandomization. We recall the following security game:

Game uf-cma-rk_{RSig}:

- **Setup Phase:** The challenger initializes the list $\text{SigList} \leftarrow \{\epsilon\}$ and samples a pair of keys $(pk, sk) \leftarrow \text{RSig.Gen}(\text{par})$. Then the public key pk is sent to the adversary $\mathcal{A}$.

- **Online Phase:** $\mathcal{A}$ is given access to a signing oracle RSig which works as follows. On input a message m and a randomness ρ , derive a pair of keys rerandomized with the randomness ρ , as $\text{sk}' \leftarrow \text{RSig.SKDer}(\text{sk}, \rho)$ and $\text{pk}' \leftarrow \text{RSig.PKDer}(\text{pk}, \rho)$. A signature is then derived on message m under the secret key sk' as $\sigma \leftarrow \text{RSig.Sign}(\text{sk}', m)$. The message m is stored in the SigList and eventually the signature σ is returned as the answer.
- **Output Phase:** Finally, the adversary $\mathcal{A}$ wins the game if it can provide a signature σ^* for a message m^* relative to randomness ρ^* , where the following holds: (1) the message m^* has not been queried before, i.e., $m^* \notin \text{SigList}$, and (2) σ^* is a valid forgery, i.e., $\text{RSig.Verify}(\text{pk}^*, \sigma^*, m^*) = 1$, where $\text{pk}^* \leftarrow \text{RSig.PKDer}(\text{pk}, \rho^*)$.

For an algorithm $\mathcal{A}$ we define $\mathcal{A}$'s advantage in game $\text{uf-cma-rk}_{\text{RSig}}$ as $\text{Adv}_{\text{uf-cma-rk}_{\text{RSig}}}^{\mathcal{A}} = \Pr[\text{uf-cma-rk}_{\text{RSig}}^{\mathcal{A}} = 1]$.

We now show the proof of Theorem 5.4 as presented in Section 5. Concretely, we show that for any signature scheme with rerandomizable keys RSig that satisfies uf-cma-rk security and for any generic transformation from RSig to a hierarchical deterministic wallet scheme $\text{HDWal}^{\text{RSig}}$ there exists no reduction from $\text{uf-cma-rk}_{\text{RSig}}$ to $\text{wufcma1}_{\text{HDWal}^{\text{RSig}}}$ that does not incur a loss polynomial in the number of SKLeak0 oracle queries q_{sk} . In particular, this shows that the reduction in our proof of Theorem 5.3 is optimal and cannot be improved even assuming a generic transformation $\text{HDWal}^{\text{RSig}}$ from a uf-cma-rk secure signature scheme with rerandomizable keys. We show this result by assuming a reduction $\mathcal{R}$ that reduces $\text{uf-cma-rk}_{\text{RSig}}$ to $\text{wufcma1}_{\text{HDWal}^{\text{RSig}}}$ and by providing a metareduction $\mathcal{M}$ that uses $\mathcal{R}$ to win its own $\text{uf-cma-rk}_{\text{RSig}}$ game. We show that the advantage of $\mathcal{M}$ in game $\text{uf-cma-rk}_{\text{RSig}}$ has a polynomial loss in the number of SKLeak0 oracle queries q_{sk} . Our proof proceeds in a similar fashion as the proofs in [23, Theorem 2] and [13, Theorem 4].

We now provide the full formal proof of Theorem 5.4.

PROOF. We describe a metareduction $\mathcal{M}$ that plays in the game $\text{uf-cma-rk}_{\text{RSig}}^{\mathcal{M}}$ and simulates the game $\text{uf-cma-rk}_{\text{RSig}}^{\mathcal{R}}$ to $\mathcal{R}$. Additionally, $\mathcal{M}$ simulates an adversary in game $\text{wufcma1}_{\text{HDWal}^{\text{RSig}}}$ to $\mathcal{R}$. $\mathcal{M}$ receives a public key $\text{pk}_{\mathcal{M}}$ from its challenger, and access to a signing oracle RSig . The goal of $\mathcal{M}$ is to come up with a valid forgery in the $\text{uf-cma-rk}_{\text{RSig}}^{\mathcal{M}}$ game. The metareduction proceeds as follows:

- (1) $\mathcal{M}$ runs the reduction $\mathcal{R}$ with public key $\text{pk}_{\mathcal{M}}$ as input and simulates game $\text{uf-cma-rk}_{\text{RSig}}^{\mathcal{R}}$ to $\mathcal{R}$ by simply forwarding $\mathcal{R}$'s queries to its own challenger. $\mathcal{R}$ sends a public key pk to $\mathcal{M}$ in the game $\text{wufcma1}_{\text{HDWal}^{\text{RSig}}}$.
- (2) Assume that $\mathcal{M}$ in game $\text{wufcma1}_{\text{HDWal}^{\text{RSig}}}$ has made q queries to the HChild0 oracle on input pairs $(\text{addr}_i, \cdot, e \cdot)$ and let X be a set consisting of the q addresses that $\mathcal{M}$ has queried the HChild0 oracle on (for simplicity we write $X = \{\text{addr}_1, \dots, \text{addr}_q\}$). Let $q_{\text{sk}} \leq \lfloor q/2 \rfloor$ be the number of addresses, for which $\mathcal{M}$ invokes the Secret Key Leakage oracle. $\mathcal{M}$ picks $i \xleftarrow{\$} \{1, \dots, q_{\text{sk}}\}$, chooses $\text{addr}^* \xleftarrow{\$} X$ and $(\text{addr}_1, \dots, \text{addr}_{q_{\text{sk}}}) \xleftarrow{\$} (X \setminus \{\text{addr}^*\})^{q_{\text{sk}}}$. This defines the following two sequences:

$$X_s := (\text{addr}_1, \dots, \text{addr}_{i-1}, \text{addr}^*)$$

$$X'_s := (\text{addr}_1, \dots, \text{addr}_{q_{\text{sk}}})$$

- (3) $\mathcal{M}$ queries the SKLeak0 oracle on addresses in the set X_s and receives the corresponding secret keys as answers from $\mathcal{R}$. In particular, since $\text{addr}^* \in X_s$, $\mathcal{M}$ knows the secret key sk^* .
- (4) $\mathcal{R}$ is then rewound to the initial state. Then $\mathcal{M}$, in game $\text{wufcma1}_{\text{HDWal}^{\text{RSig}}}$, queries the SKLeak0 oracle on addresses from the set X'_s . Since $\text{addr}^* \notin X'_s$, $\mathcal{M}$ has not corrupted the node with addr^* .
- (5) $\mathcal{M}$ now tosses a biased coin τ with probability $\epsilon_{\mathcal{A}}$ of outputting 1. If $\tau = 0$, $\mathcal{M}$ sends $\perp$ to $\mathcal{R}$ in the $\text{wufcma1}_{\text{HDWal}^{\text{RSig}}}$ game. If $\tau = 1$, $\mathcal{M}$ samples a random message m , creates a signature σ on m with secret key sk^* and returns (σ, m) as a valid forgery. This execution is done in time $t_{\mathcal{A}}$ such that $\mathcal{M}$ correctly simulates an adversary in game $\text{wufcma1}_{\text{HDWal}^{\text{RSig}}}$.
- (6) Since $\mathcal{R}$ was rewound, sk^* was not revealed and (σ, m) constitutes a valid forgery. $\mathcal{R}$ derives a signature (σ', m') corresponding to challenge key $\text{pk}_{\mathcal{M}}$ and returns it to $\mathcal{M}$. $\mathcal{M}$ can return (σ', m') to the $\text{uf-cma-rk}_{\text{RSig}}^{\mathcal{M}}$ game.

Success probability of $\mathcal{M}$. We now analyze the probability with which $\mathcal{M}$ can win the $\text{uf-cma-rk}_{\text{RSig}}^{\mathcal{M}}$ game. Let Q be a set of sequences of addresses such that for any sequence $(\text{addr}_1, \dots, \text{addr}_j) \in Q$, the corresponding SKLeak0 oracle queries are answered correctly by $\mathcal{R}$. Additionally, it holds that if $(\text{addr}_1, \dots, \text{addr}_j) \in Q$, then also $(\text{addr}_1, \dots, \text{addr}_{j-1}) \in Q$. Let us now consider a (possibly unbounded) real adversary $\mathcal{A}$ (i.e., $\mathcal{A}$ is not simulated by $\mathcal{M}$), who issues queries to the SKLeak0 oracle on inputs $\text{addr}_j \in X'_s$ and eventually outputs a valid forgery (σ, m) with success probability $\epsilon_{\mathcal{A}}$. The view of $\mathcal{R}$ is exactly the same when interacting with the real adversary $\mathcal{A}$ or with the adversary who is simulated by $\mathcal{M}$ (which we denote by $\mathcal{A}_{\mathcal{M}}$) except if the following bad event occurs: $X_s \notin Q$ but $X'_s \in Q$. In this case, the reduction $\mathcal{R}$ did not answer all SKLeak0 oracle queries correctly in the interaction with $\mathcal{A}_{\mathcal{M}}$ before the rewind but did so after the rewind. If this event occurs, the real adversary $\mathcal{A}$ would output a valid forgery, while the simulated adversary $\mathcal{A}_{\mathcal{M}}$ would not. Hence, the reduction $\mathcal{R}$ would be able to distinguish the real from the simulated execution.

Let $\mathcal{R}^{\mathcal{A}}$ and $\mathcal{R}^{\mathcal{A}_{\mathcal{M}}}$ denote the execution of the reduction w.r.t. the real and simulated adversary, respectively. The executions $\mathcal{R}^{\mathcal{A}}$ and $\mathcal{R}^{\mathcal{A}_{\mathcal{M}}}$ are identical, except if the following bad events occur in $\mathcal{R}^{\mathcal{A}_{\mathcal{M}}}$: $X'_s \in Q$ and $X_s \notin Q$ and $\tau = 1$. Therefore, we get:

$$\begin{aligned} & |\Pr[(\sigma', m') \leftarrow \mathcal{R}^{\mathcal{A}_{\mathcal{M}}}(\text{pk}_{\mathcal{M}}) \wedge (\sigma' \text{ is valid on } m')] \\ & - \Pr[(\sigma', m') \leftarrow \mathcal{R}^{\mathcal{A}}(\text{pk}_{\mathcal{M}}) \wedge (\sigma' \text{ is valid on } m')]| \\ & \leq \epsilon_{\mathcal{A}} \cdot \Pr[X'_s \in Q \wedge X_s \notin Q]. \end{aligned}$$

We recall the following lemma due to Coron [13].

Lemma D.1 *Let Q be a set of sequences of at most q_{sk} integers in X , such that for any sequence $(\text{addr}_1, \dots, \text{addr}_j) \in Q$, we have $(\text{addr}_1, \dots, \text{addr}_{j-1}) \in Q$. Then:*

$$\Pr_{i \xleftarrow{\$} \{1, \dots, q_{\text{sk}}\}} \left[\begin{array}{l} (\text{addr}_1, \dots, \text{addr}_{q_{\text{sk}}}) \in Q \\ \wedge (\text{addr}_1, \dots, \text{addr}_{i-1}, \text{addr}^*) \notin Q \end{array} \right] \leq \epsilon_{\mathcal{A}} \cdot \Pr[X'_s \in Q \wedge X_s \notin Q]$$

$$\leq \frac{\exp(-1)}{q_{sk}}.$$

From lemma D.1, representing addresses as integers, we get that

$$\Pr[\mathcal{X}'_s \in Q \wedge \mathcal{X}_s \notin Q] \leq \frac{\exp(-1)}{q_{sk}} \left(1 - \frac{q_{sk}}{q}\right)^{-1}.$$

Note that the additional term $\left(1 - \frac{q_{sk}}{q}\right)^{-1}$ comes from the fact that we chose all addr_i from the set $\mathcal{X} \setminus \{\text{addr}^*\}$ instead of $\mathcal{X}$. Hence, we need to consider the probability that for all addr_i it holds that $\text{addr}_i \neq \text{addr}^*$.

From this, we obtain the success probability $\text{Adv}_{\text{uf-cma-rkRSig}}^{\mathcal{M}}$ for $\mathcal{M}$ as follows:

$$\begin{aligned} \text{Adv}_{\text{uf-cma-rkRSig}}^{\mathcal{M}} &= \Pr[(\sigma', m') \leftarrow \mathcal{R}^{\mathcal{A}, \mathcal{M}}(\text{pk}_{\mathcal{M}}) \wedge (\sigma' \text{ is valid on } m')] \\ &\geq \Pr[(\sigma', m') \leftarrow \mathcal{R}^{\mathcal{A}}(\text{pk}_{\mathcal{M}}) \wedge (\sigma' \text{ is valid on } m')] \\ &\quad - \epsilon_{\mathcal{A}} \cdot \frac{\exp(-1)}{q_{sk}} \left(1 - \frac{q_{sk}}{q}\right)^{-1} \\ &\geq \Pr[(\sigma', m') \leftarrow \mathcal{R}^{\mathcal{A}}(\text{pk}_{\mathcal{M}}) \wedge (\sigma' \text{ is valid on } m')] \\ &\quad - \epsilon_{\mathcal{A}} \cdot \frac{2 \exp(-1)}{q_{sk}} \\ &\geq \epsilon_{\mathcal{R}}(\epsilon_{\mathcal{A}}) - \epsilon_{\mathcal{A}} \cdot \frac{2 \exp(-1)}{q_{sk}} \end{aligned}$$

Note that, since $\mathcal{M}$ rewinds the reduction $\mathcal{R}$ once, the running time of $\mathcal{M}$ can be upper bounded by $t_{\mathcal{M}} \leq 2 \cdot t_{\mathcal{R}}(t_{\mathcal{A}})$. ■

E DISCUSSION (CONTD.)

The following Theorem follows from Theorems 3.3 and 5.3.

Theorem E.1 Let $H_0: \{0, 1\}^* \rightarrow \mathbb{Z}_p$, $H_1: \{0, 1\}^* \rightarrow \mathbb{Z}_p$ be a hash function modeled as a random oracle. Let $\text{REC}[H_1]$ be the scheme as defined in Figure 7. Let $\text{HDWal}[\text{RSig}]$ be the construction as defined in Figure 5. We define $\text{HDWal}[\text{REC}[H_1]]$ as the construction of $\text{HDWal}[\text{RSig}]$, instantiated with $\text{RSig} = \text{REC}[H_1]$. Let $\mathcal{A}$ be an algorithm that plays in the game $\text{wufcma1}_{\text{HDWal}[\text{REC}[H_1]]}$, then there exists an algorithm $\mathcal{C}$ running in roughly the same time as $\mathcal{A}$ such that

$$\text{Adv}_{\text{uf-cma1EC}[H_0]}^{\mathcal{C}} \geq \left(\frac{1}{4e(q_{sk} + 1)} \cdot \text{Adv}_{\text{wufcma1HDWal}[\text{RSig}]}^{\mathcal{A}} - \frac{q_{H_1}^2}{p} \right) \cdot \frac{1}{q},$$

where q_{H_1} is the number of random oracle queries, q is the total number of HChild0 and NHChild0 oracle queries and q_{sk} is the number of queries to the SKLeak0 oracle.

The following Theorem follows from Theorem 5.3 and Theorem 5.1 from [15].

Theorem E.2 Let $H_0: \{0, 1\}^* \rightarrow \mathbb{Z}_p$, $H_1: \{0, 1\}^* \rightarrow \mathbb{Z}_p$ be hash functions modeled as a random oracles. Let $\text{REC}'[H_1]$ be the scheme as defined in Figure 6. Let $\text{HDWal}[\text{RSig}]$ be the construction as defined in Figure 5. We define $\text{HDWal}[\text{REC}'[H_1]]$ as the construction of $\text{HDWal}[\text{RSig}]$, instantiated with $\text{RSig} = \text{REC}'[H_1]$. Let $\mathcal{A}$ be an algorithm that plays in the game $\text{wufcma1}_{\text{HDWal}[\text{REC}'[H_1]]}$, then there exists an algorithm $\mathcal{C}$ running in roughly the same time as $\mathcal{A}$ such that

$$\text{Adv}_{\text{uf-cma1EC}[H_0]}^{\mathcal{C}} \geq \frac{1}{4e(q_{sk} + 1)} \cdot \text{Adv}_{\text{wufcma1HDWal}[\text{RSig}]}^{\mathcal{A}} - \frac{3q_{H_1}^2}{p},$$

Algorithm $\text{REC}'[H_1].\text{Sign}(\text{sk}, m)$	Algorithm $\text{REC}'[H_1].\text{RandSK}(\text{sk}; \rho)$
00 $\psi \xleftarrow{\$} \{0, 1\}^K$	00 $\text{sk}' \leftarrow \text{sk} \cdot \rho \bmod p$
01 $\hat{m} \leftarrow (\text{pk}, \psi, m)$	01 Return sk'
02 $\sigma' \leftarrow \text{EC}[H_1].\text{Sign}(\text{sk}, \hat{m})$	Algorithm $\text{REC}'[H_1].\text{RandPK}(\text{pk}; \rho)$
03 Return $\sigma = (\psi, \sigma')$	02 $\text{pk}' \leftarrow \text{pk} \cdot \rho$
Algorithm $\text{REC}'[H_1].\text{Verify}(\text{pk}, \sigma, m)$	03 Return pk'
04 $(\psi, \sigma') \leftarrow \sigma$	
05 $\hat{m} \leftarrow (\text{pk}, \psi, m)$	
06 Return $\text{EC}[H_1].(\text{pk}, \sigma', \hat{m})$	

Figure 6: Salted and key-prefixed version of the ECDSA signature scheme with perfectly rerandomizable keys $\text{REC}'[H_1] := (\text{REC}'[H_1].\text{Gen} = \text{EC}[H_1].\text{Gen}, \text{REC}'[H_1].\text{Sign}, \text{REC}'[H_1].\text{Verify}, \text{REC}'[H_1].\text{RandSK}, \text{REC}'[H_1].\text{RandPK})$ from the ECDSA signature scheme $\text{EC}[H_1]$. $H_1: \{0, 1\}^* \rightarrow \mathbb{Z}_p$ denotes a hash function.

Algorithm $\text{MREC}[H_1].\text{Sign}(\text{sk}, m)$	Algorithm $\text{MREC}[H_1].\text{RandSK}(\text{sk}; \rho)$
00 $\text{pm} \leftarrow (\text{pk}, m)$	00 $\text{sk}' \leftarrow \text{sk} \cdot \rho \bmod p$
01 $\sigma \leftarrow \text{EC}[H_1].\text{Sign}(\text{sk}, \text{pm})$	01 Return sk'
02 Return σ	Algorithm $\text{MREC}[H_1].\text{RandPK}(\text{pk}; \rho)$
Algorithm $\text{MREC}[H_1].\text{Verify}(\text{pk}, \sigma, m)$	02 $\text{pk}' \leftarrow \text{pk} \cdot \rho$
03 $\text{pm} \leftarrow (\text{pk}, m)$	03 Return pk'
04 Return $\text{EC}[H_1].\text{Verify}(\text{pk}, \sigma', \text{pm})$	

Figure 7: Salt-free and key-prefixed version of the ECDSA signature scheme with perfectly rerandomizable keys $\text{MREC}[H_1] := (\text{MREC}[H_1].\text{Gen} = \text{EC}[H_1].\text{Gen}, \text{MREC}[H_1].\text{Sign}, \text{MREC}[H_1].\text{Verify}, \text{MREC}[H_1].\text{RandSK}, \text{MREC}[H_1].\text{RandPK})$ from the ECDSA signature scheme $\text{EC}[H_1]$. $H_1: \{0, 1\}^* \rightarrow \mathbb{Z}_p$ denotes a hash function.

where q_{H_1} is the number of random oracle queries and q_{sk} is the number of queries to the SKLeak0 oracle.

Lemma E.3 Consider the algorithm $\text{Trf}[H_0, H_1]_{\text{EC}}$ in Figure 8. Suppose that:

- $\omega = \frac{H_1(m_1)}{H_0(m_0)} \in \mathbb{Z}_p$,
- $X_0, X_1 \in \mathbb{B}$ s.t. $X_0 = x_0 \cdot G$ and $X_1 = \omega \cdot X_0$,
- $\text{EC}[H_1].\text{Verify}(X_1, \sigma_1, m_1) = 1$,
- $\sigma_0 \leftarrow \text{Trf}[H_0, H_1]_{\text{EC}}(m_0, m_1, \sigma_1, \omega, X_0, X_1)$.

Then $\text{EC}[H_0].\text{Verify}(X_0, \sigma_0, m_0) = 1$.

$\text{Trf}[H_0, H_1]_{\text{EC}}(m_0, m_1, \sigma_1, \omega, X_0, X_1)$
00 $z_0 \leftarrow H_0(m_0)$
01 $z_1 \leftarrow H_1(m_1)$
02 If $(\text{EC}[H_1].\text{Verify}(\sigma_1, m_1, X_1) = 0) \vee (\omega \neq \frac{z_1}{z_0} \vee X_1 \neq X_0 \cdot \omega)$:
03 Return $\perp$
04 $(r, s_1) \leftarrow \sigma_1$
05 $s_0 \leftarrow \frac{z_1}{\omega} \bmod p$
06 $\sigma_0 \leftarrow (r, s_0)$
07 Return σ_0

Figure 8: Figure shows the $\text{Trf}_{\text{ECDSA}}$ algorithm for hash functions $H_0, H_1: \{0, 1\}^* \rightarrow \mathbb{Z}_p$.

Theorem E.4 Let $H_0: \{0, 1\}^* \rightarrow \mathbb{Z}_p$, $H_1: \{0, 1\}^* \rightarrow \mathbb{Z}_p$ be a hash function modeled as a random oracle. Let $\mathcal{A}$ be an algorithm that plays in the game $\text{uf-cma-hrk}_{\text{MREC}[H_1]}$, then there exists an algorithm C running in roughly the same time as $\mathcal{A}$ such that

$$\text{Adv}_{\text{uf-cma-EC}[H_0]}^C \geq \text{Adv}_{\text{uf-cma-hrk}_{\text{MREC}[H_1]}}^{\mathcal{A}} - \frac{3q^2}{p}$$

PROOF. Consider an adversary $\mathcal{A}$ playing in Game $\text{uf-cma-hrk}_{\text{MREC}[H_1]}$. As such $\mathcal{A}$ is granted access to the oracles Rand , RSign , and the random oracle $H_1: \{0, 1\}^* \rightarrow \mathbb{Z}_p$. In the following, we use that $2^K \leq p$. We prove the statement via a sequence of games. Each game $\mathbf{G}_{i(i>0)}$ is presented in Figure 10 via the description of the oracles that are modified with respect to the previous game $\mathbf{G}_{i-1}$. The exact differences of game $\mathbf{G}_i$ to game $\mathbf{G}_{i-1}$ are highlighted in the form of boxed pseudocode. Moreover, we denote by $E_{i-1,i}$ a difference event, where the indices of the event correspond to games $\mathbf{G}_{i-1}, \mathbf{G}_i$ that are affected by the event.

GAME $\mathbf{G}_0$: This game is equivalent to the original $\text{uf-cma-hrk}_{\text{MREC}[H_1]}$ game. In particular, a key pair (sk, pk) is sampled as $(sk, pk) \xleftarrow{\$} \text{MREC}[H_1].\text{Gen}(\text{par})$. The adversary $\mathcal{A}$ is given pk as the challenge public key and oracle access to Rand , RSign and random oracle H_1 . $\mathcal{A}$ can query Rand to receive a randomness ρ and make a follow-up query to RSign to receive a signature on message m with respect to the rerandomized key $pk' \leftarrow pk \cdot \rho$. In particular, $\mathcal{A}$ is allowed to query RSign on every input pair (m, ρ) at most once. Additionally, $\mathcal{A}$ can make direct queries to the random oracle H_1 . The game internally maintains a random oracle H_0 in a straightforward manner, by storing a list H_0 of query/response pairs. Eventually, in order to win the game, $\mathcal{A}$ has to come up with a valid forgery σ^* on a new message m^* with respect to a randomness ρ^* . Since $\mathbf{G}_0$ proceeds as $\text{uf-cma-hrk}_{\text{MREC}[H_1]}$ we have that $\Pr[\mathbf{G}_0 = 1] = \Pr[\text{uf-cma-hrk}_{\text{MREC}[H_1]} = 1] = \text{Adv}_{\text{uf-cma-hrk}_{\text{MREC}[H_1]}}^{\mathcal{A}}$.

GAME $\mathbf{G}_1$: In $\mathbf{G}_1$, the way that random oracle queries to H_1 from $\mathcal{A}$ are answered, is internally modified as follows. To answer queries to H_1 , $\mathbf{G}_1$ internally keeps two lists H_1 and H'_1 which it programs throughout its interaction with $\mathcal{A}$. Depending on whether a queried message m contains as part of its prefix a public key pk' , it programs $H_1[m]$ and $H'_1[m]$ in two different possible ways. Note that pk' is the result of rerandomizing pk as $pk' = pk \cdot \rho$, where $\rho \leftarrow \text{Rand}(\rho \in \text{RList})$ is a previous answer to an oracle query Rand . We now analyze the three types of queries to H_1 that can occur.

- $H_1[m] \neq \perp$: In this case, $\mathbf{G}_1$ returns $H_1[m]$.
- $H_1[m] = \perp$ and m is of the form $m = (pk', \cdot)$, s.t. $pk' = pk \cdot \rho$ for some $\rho \in \text{RList}$: In this case, $\mathbf{G}_1$ computes $h \leftarrow H_0(ctr)$, where $ctr \xleftarrow{\$} \{0, 1\}^K$. Consequently, $\mathbf{G}_1$ sets $H_1[m] \leftarrow \rho \cdot h \bmod p$ and $H'_1[m] \leftarrow ctr$. It returns $H_1[m]$.
- Otherwise, $\mathbf{G}_1$ samples $h \xleftarrow{\$} \mathbb{Z}_p$ and sets the values $H_1[m] \leftarrow h$, $H'_1[m] \leftarrow \epsilon$. It then returns $H_1[m]$.

It is easy to see that all answers for queries to H_1 that $\mathbf{G}_1$ returns are uniformly distributed from $\mathcal{A}$'s perspective. This follows from the uniformity of output h computed via random oracle H_0 . Therefore, $\mathbf{G}_1$ behaves exactly as $\mathbf{G}_0$.

GAME $\mathbf{G}_2$: In $\mathbf{G}_2$, the way in which queries to Rand are answered, is internally modified as follows. When $\mathcal{A}$ asks a query of the

Game $\mathbf{G}_0$ 00 RList $\leftarrow \{\epsilon\}$ 01 bad $\leftarrow \text{false}$ 02 $(sk, pk) \xleftarrow{\$} \text{MREC}[H_1].\text{Gen}(\text{par})$ 03 $(m^*, \sigma^*, \rho^*) \xleftarrow{\$} \mathcal{A}^{H_1, \text{Rand}, \text{RSign}}(pk)$ 04 $pk^* \leftarrow pk \cdot \rho^*$ 05 If $\rho^* \in \text{SigList}$: bad $\leftarrow \text{true}$ 06 If $\rho^* \notin \text{RList}$: bad $\leftarrow \text{true}$ 07 $b \leftarrow \text{MREC}[H_1].\text{Verify}(pk^*, \sigma^*, m^*)$ 08 Return $b \wedge \neg \text{bad}$	Oracle $\text{RSign}(m, \rho)$ 12 If $\rho \notin \text{RList}$: Return $\perp$ 13 $pk' \leftarrow pk \cdot \rho \bmod p$ 14 $sk' \leftarrow sk \cdot \rho \bmod p$ 15 $pm \leftarrow (pk', m)$ 16 $\sigma \leftarrow \text{MREC}[H_1].\text{Sign}(pm, sk')$ 17 SigList $\leftarrow \text{SigList} \cup \{pm\}$ 18 Return σ
Oracle Rand 09 $\rho \xleftarrow{\$} \mathcal{R}$ 10 RList $\leftarrow \text{RList} \cup \{\rho\}$ 11 Return ρ	Oracle $H_1(m)$ 19 If $H_1[m] \neq \perp$ 20 Return $H_1[m]$ 21 $H_1[m] \xleftarrow{\$} \mathbb{Z}_p$ 22 Return $H_1[m]$
	$H_0[m]$ 23 If $H_0[m] \neq \perp$ 24 Return $H_0[m]$ 25 $H_0[m] \xleftarrow{\$} \mathbb{Z}_p$ 26 Return $H_0[m]$

Figure 9: Game $\mathbf{G}_0 = \text{uf-cma-hrk}_{\text{MREC}[H_0]}$ with adversary C .

Oracle $H_1(m)$ in $\mathbf{G}_1$ 00 If $H_1[m] \neq \perp$ 01 Return $H_1[m]$ 02 Parse m as $(pk', \cdot)$ 03 If $\exists \rho \in \text{RList} : pk' = pk \cdot \rho$ 04 $ctr \leftarrow \{0, 1\}^K$ 05 $h \leftarrow H_0(ctr)$ 06 $H_1[m] \leftarrow \rho \cdot h \bmod p$ 07 $H'_1[m] \leftarrow ctr$ 08 Else 09 $h \xleftarrow{\$} \mathbb{Z}_p$ 10 $H_1[m] \leftarrow h$ 11 $H'_1[m] \leftarrow \epsilon$ 12 Return $H_1[m]$	Oracle $\text{RSign}(m, \rho)$ in $\mathbf{G}_3$ 19 If $\rho \notin \text{RList}$: Return $\perp$ 20 $pk' \leftarrow pk \cdot \rho$ 21 $pm \leftarrow (pk', m)$ 22 If $H'_1[pm] = \perp$ 23 Query $H_1(pm)$ 24 $m' \leftarrow H'_1[pm]$ 25 $\sigma' \leftarrow \text{EC}[H_0].\text{Sign}(sk, m')$ 26 $\sigma \leftarrow \text{Trf}[H_0, H_1]_{\text{EC}}(pm, m', \sigma', \rho^{-1}, pk', pk)$ 27 SigList $\leftarrow \text{SigList} \cup \{pm\}$ 28 Return σ
Oracle Rand in $\mathbf{G}_2$ 13 $\rho \xleftarrow{\$} \mathcal{R}$ 14 $pk' \leftarrow pk \cdot \rho$ 15 $\forall m = (pk', \cdot)$: 16 If $H'_1[m] = \epsilon$: Abort 17 RList $\leftarrow \text{RList} \cup \{\rho\}$ 18 Return ρ	main in $\mathbf{G}_4$ 29 RList $\leftarrow \{\epsilon\}$ 30 bad $\leftarrow \text{false}$ 31 $(sk, pk) \xleftarrow{\$} \text{MREC}[H_1].\text{Gen}(\text{par})$ 32 $(m^*, \sigma^*, \rho^*) \xleftarrow{\$} \mathcal{A}^{H_1, \text{Rand}, \text{RSign}}(pk)$ 33 $pk^* \leftarrow pk \cdot \rho^*$ 34 $pm^* \leftarrow (pk^*, m^*)$ 35 If $H'_1[pm^*] = \epsilon$: Abort 36 If $pm^* \in \text{SigList}$: bad $\leftarrow \text{true}$ 37 If $\rho^* \notin \text{RList}$: bad $\leftarrow \text{true}$ 38 $b \leftarrow \text{MREC}[H_1].\text{Verify}(pk^*, \sigma^*, m^*)$ 39 Return $b \wedge \neg \text{bad}$

Figure 10: Games $\mathbf{G}_1$ – $\mathbf{G}_4$

form Rand , the game aborts if there exists a message of the form $m = (pk', \cdot)$ for which $H'_1[m]$ evaluates to ϵ and where pk' is the (rerandomized) key that corresponds to the return value ρ of Rand , i.e., $pk' = pk \cdot \rho$. The following claim bounds the probability of such an abort scenario.

Claim E.5 Let $E_{1,2}$ denote the event that $\mathbf{G}_2$ aborts during a Rand query, for which $H'_1[m]$ evaluates to ϵ , where $m = (pk', \cdot)$. Then $\Pr[E_{1,2}] \leq \frac{q^2}{p}$.

PROOF. During any particular call to the oracle Rand , this event can only occur if $\mathcal{A}$ has already made a query of the form $H_1(m)$,

where $m = (pk', \cdot)$ (prior to the oracle Rand returning the value ρ for this query). Since RList contains at most q values at any point during the game, any of them coincide with the (uniformly chosen) value ρ with probability at most $\frac{q}{p}$. Since keys are uniquely rerandomizable, a query of the form $H_1(m)$ thus also has probability at most $\frac{q}{p}$ of having been made prior to this particular call to Rand. Since there at most q queries to Rand, it follows that $\Pr[E_{1,2}] \leq \frac{q^2}{p}$.

■ Since the games $\mathbf{G}_1, \mathbf{G}_2$ are equivalent unless the event $\Pr[E_{1,2}]$ occurs, $\Pr[\mathbf{G}_0 = 1] \leq \Pr[\mathbf{G}_1] + \frac{q^2}{p}$

GAME $\mathbf{G}_3$: In $\mathbf{G}_3$, the way that signing queries from $\mathcal{A}$ are answered, is again internally modified as follows. When $\mathcal{A}$ makes a query of the form $\text{RSign}(m, \rho)$, $\mathbf{G}_3$ first checks whether $\rho \in \text{RList}$ and if not, returns $\perp$. Otherwise, it computes $pk' \leftarrow pk \cdot \rho$, and sets $pm \leftarrow (pk', m)$. If $H_1[\hat{m}] = \perp$, it internally queries H_1 on input message pm . This means it queries $h \leftarrow H(ctr)$, where $ctr \xleftarrow{\$} \{0, 1\}^k$. $\mathbf{G}_3$ internally sets $H_1[pm] \leftarrow \rho \cdot h \bmod p$ and stores $H'_1[pm] \leftarrow ctr$. After making the query to H_1 , $\mathbf{G}_3$ fetches $m' \leftarrow H'_1[pm]$, where m' was set to ctr during H_1 query. Since sk is known to the game, it can now compute the signature σ' as $\sigma' \xleftarrow{\$} \text{EC}[H_0].\text{Sign}(sk, m')$. Finally, it computes and returns the signature σ as $\sigma \leftarrow \text{Trf}[H_0, H_1]_{\text{EC}}(pm, m', \sigma', \rho^{-1}, pk', pk)$, where $pk = pk' \cdot \rho^{-1}$.

Claim E.6 $\Pr[\mathbf{G}_2 = 1] = \Pr[\mathbf{G}_3 = 1]$

PROOF. We argue that in both games, the answers to signing queries are identically distributed. To this end, we analyze how $\mathbf{G}_3$ replies to a query of the form $\text{RSign}(m, \rho)$. $\mathbf{G}_3$ derives signature σ on input (m, ρ) as $\sigma \leftarrow \text{Trf}[H_0, H_1]_{\text{EC}}(pm, m', \sigma', \rho^{-1}, pk', pk)$, where $m' = H'_1[pm]$, $pk = pk' \cdot \rho^{-1}$, $\text{EC}[H_0].\text{Verify}(pk, \sigma', m') = 1$, and $\frac{H_0[m']}{H_1[pm]} = \frac{h'}{H_1[pm]} = \frac{h'}{\rho \cdot h} = \rho^{-1} \bmod p$. It follows from Lemma E.3 that σ constitutes a correct signature on message pm and under public key pk' relative to $\text{EC}[H_0].\text{Verify}$. It follows immediately that the signature σ constitutes a valid signature relative to $\text{MREC}[H_1].\text{Verify}$. This concludes the proof. ■

GAME $\mathbf{G}_4$: $\mathbf{G}_4$ behaves identically to $\mathbf{G}_3$ except for the following modification in the main procedure: Upon receiving a forgery of the form (m^*, σ^*, ρ^*) from $\mathcal{A}$, it sets $pm^* \leftarrow (pk^*, m^*)$ and aborts if $H'_1[pm^*] = \epsilon$.

Claim E.7 Let $E_{3,4}$ be the event that $\mathbf{G}_4$ aborts if $H'_1[pm^*] = \epsilon$, where $pm^* = (pk^*, m^*)$. Then $\Pr[E_{3,4}] \leq \frac{q^2}{p}$.

PROOF. The only way this event can happen, is if $\mathcal{A}$ manages to make a query of the form $H_1(pm^*)$ before querying Rand to obtain the corresponding value of ρ^* . The proof of this claim follows in a similar way as the corresponding proof in claim E.5. ■

Since the games $\mathbf{G}_3, \mathbf{G}_4$ are equivalent unless event $E_{3,4}$ occurs, $\Pr[\mathbf{G}_3 = 1] \leq \Pr[\mathbf{G}_4 = 1] + \frac{q^2}{p}$

Reduction to UF-CMA security. We describe an algorithm C that plays in the $\text{uf-cma}_{\text{EC}[H_0]}$ game and simulates game $\mathbf{G}_4$ to $\mathcal{A}$. Instead of sampling its own key pair as is done in $\mathbf{G}_4$, C obtains as input a public key pk_C from the $\text{uf-cma}_{\text{EC}[H_0]}$ game and is given access to the signing oracle Sign to obtain signatures under pk_C

under messages of its choice. Furthermore, C has access to the random oracle H_0 by which it replaces the list H_0 . C runs $\mathcal{A}$ on input pk_C .

Simulation of Randomness Queries. Queries to Rand from $\mathcal{A}$ do not require knowledge of the secret key corresponding to pk_C and hence are straight forward to simulate.

Simulation of Random Oracle Queries. C 's simulation of random oracle queries coincides with the above programming strategy that is already internally present in $\mathbf{G}_4$.

Simulation of Signing Queries. Recall that in $\mathbf{G}_4$, queries of the form $\text{RSign}(m, \rho)$ internally prompt the computation of signature $\sigma' = \text{EC}[H_0].\text{Sign}(sk_C, m')$, where $m' \leftarrow ctr$. Since C does not know sk_C , it needs to compute σ' via a call to its signing oracle, i.e., as $\sigma' \leftarrow \text{Sign}(m')$. Other than that C simulates such a query exactly as internally done for $\mathbf{G}_4$.

EXTRACTING THE FORGERY. When the tuple (m^*, σ^*, ρ^*) is returned as an answer from $\mathcal{A}$, C checks whether it constitutes a valid forgery, and aborts otherwise (note that in this case, $\mathbf{G}_4$ would return 0, so C can safely abort). In case C does not abort, it computes $pk^* = pk_C \cdot \rho^*$, where pk^* is the public key under which $\mathcal{A}$'s forgery is valid. C computes $pm^* \leftarrow (pk^*, m^*)$ and if $H'_1[pm^*] = \epsilon$, it aborts. Otherwise, C fetches $m' \leftarrow H'_1[pm^*]$ and computes

$$\sigma' \leftarrow \text{Trf}[H_1, H_0]_{\text{ECDSA}}(m', pm^*, \sigma^*, \rho^*, pk_C, pk^*).$$

Since $H_1[pm^*] = H_0(H'_1[pm^*]) \cdot \rho^* = H(m') \cdot \rho^*$, we have that $\frac{H_1[pm^*]}{H_0(m')} = \frac{H_0(m') \cdot \rho^*}{H_0(m')} = \rho^*$. Together with $pk^* = pk_C \cdot \rho^*$ and $\text{MREC}[H_1].\text{Verify}(pk^*, \sigma^*, pm^*) = 1$, Lemma E.3 implies that

$$\text{EC}[H_0].\text{Verify}(pk_C, \sigma', m') = 1.$$

Claim E.8 (m', σ') constitutes a valid forgery in $\text{uf-cma}_{\text{EC}[H_0]}$ with probability $1 - q^2/p$.

PROOF. We have to show that the query $\text{Sign}(m')$ was not made by C during its simulation and hence (m', σ') is a valid forgery in $\text{uf-cma}_{\text{EC}[H_0]}$. Note that $\mathcal{A}$ has not made a query of the form $\text{RSign}(m^*, \rho^*)$ throughout the simulation. If it had, (m^*, σ^*, ρ^*) would not constitute a valid forgery in $\mathbf{G}_4$ and the simulation would have aborted at this point. This implies that C never had to simulate a query $\text{RSign}(m^*, \rho^*)$ to $\mathcal{A}$ which entailed a H_1 query on message $pm^* \leftarrow (pk^*, m^*)$. Hence, m' associated with query $H_1[pm^*]$ was not queried by C to the oracle Sign in any query of the form $\text{RSign}(m, \rho)$ with $m \neq m^*$ unless there exist (any) two values m_1, m_2 s.t. $H'_1[m_1] = H'_1[m_2] \neq \perp$. It is easy to see that this happens with probability at most q^2/p during C 's simulation, since all values that C queries to the oracle Sign are sampled independently and uniformly at random from $\{0, 1\}^k$. ■

From claims E.5-E.7, we have $\Pr[\mathbf{G}_0 = 1] \leq \Pr[\mathbf{G}_4] + \frac{3q^2}{p}$. Since C provides a perfect simulation of $\mathbf{G}_4$ to $\mathcal{A}$ up to an error of q^2/p , as shown in the previous claim, we obtain

$$\text{Adv}_{\text{uf-cma-hrk}, \text{MREC}[H_1]}^{\mathcal{A}} \leq \text{Adv}_{\mathbf{G}_4}^{\mathcal{A}} + \frac{3q^2}{p} \leq \text{Adv}_{\text{uf-cma}, \text{EC}[H_0]}^C + \frac{3q^2}{p},$$

which implies the theorem. ■

Theorem 5.3 can be combined with Theorem E.4 in a similar manner as Theorem E.2.